GarageGo

Software Requirements Specification(SRS)

Smart Garage & Emergency Vehicle Repair Platform

Item	Details
Version	v0.1
Authors	P. Kasturi SE/2022/053 K. Kajaluxmy SE/2022/056 V. Vanushan SE/2022/031 S. Krishnapiriyan SE/2022/048
Status	Draft – Sprint 2
Project	GarageGo
Date	22 May 2026
Course	SENG 31242 System Design Project
Organisation	seng31242-garagego-2026

Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the complete functional and non-functional requirements for **GarageGo**, a multi-sided digital marketplace for vehicle repair and roadside assistance in Sri Lanka. The intended audience includes the development team, project supervisors and prospective stakeholders evaluating system feasibility.

GarageGo addresses two core market failures identified through customer interviews (Issue #9) and surveys:

1. **Discovery friction for customers:** Vehicle owners struggle to find trusted, available garages quickly, especially during unexpected breakdowns. Interview participants reported relying on word-of-mouth recommendations (P01: *"I usually ask my uncle—he knows a good mechanic in Jaffna"*) and facing long, uncertain search times with no transparency on pricing or quality.
2. **Operational inefficiency for garages:** Small and medium garage owners lack digital tools for capacity management, customer acquisition and real-time dispatch. Current methods—phone calls, WhatsApp messages, and paper logbooks—lead to no-shows, empty service bays, and cash-handling risks.

GarageGo digitises the entire vehicle repair lifecycle through two core functions:

- **Scheduled Booking:** Customers search nearby garages, filter by vehicle type and services, view transparent pricing and reviews, book appointments and track service progress.
- **Emergency SOS Dispatch:** Customers trigger roadside assistance with one tap; the system broadcasts to eligible garages with emergency coverage; the first acceptance within 60 seconds wins; a mechanic is dispatched with live GPS tracking.

This document specifies all requirements necessary to guide the design, implementation, and testing of the GarageGo platform.

1.2 Scope

1.2.1 In Scope

The following components and features are within the scope of this SRS:

Component	Description
Customer Mobile Application	iOS and Android app (React Native/Expo) for vehicle registration, garage discovery, booking creation, emergency SOS, service history, and reviews
Garage Owner Management System	System for garage registration, service management, booking acceptance/rejection, mechanic assignment for repair and emergency requests, customer communication, invoice generation, and service tracking
Mechanic Mobile Application	iOS and Android app for emergency job reception, GPS navigation, live location sharing, status updates (arrived/in-progress/complete), and job history
Admin Web Dashboard	Interface for garage approval, user management, complaint resolution, refund triggering, and platform analytics
Backend API	Node.js/Express REST API with Socket.io real-time communication, JWT authentication, and webhook handlers
Location Services	Google Maps API for garage geocoding, distance-based search, emergency dispatch routing, and mechanic GPS tracking
Notification Services	Push notifications and SMS OTP verification
File Storage	Cloudinary or Amazon S3 for garage photos, vehicle images, invoice attachments and complaint evidence

1.2.2 Out of Scope

The following are explicitly excluded from this SRS:

Item	Rationale
Physical garage operations	Tool inventory, equipment maintenance, and workshop floor management are internal garage processes
Vehicle insurance claim processing	Insurance workflows require regulatory partnerships beyond MVP scope
Third-party spare parts procurement	Parts ordering and supply chain integration is future enhancement
Multi-country expansion	Platform targets Sri Lanka only for MVP; internationalisation is post-MVP
Offline-first architecture	Full offline functionality is future work; MVP includes basic caching for garage list browsing only
AI chatbot (symptom diagnosis)	Not part of core MVP requirements
IoT predictive maintenance	Sensor integration and anomaly detection are roadmap features beyond current scope
Online card payment processing	GarageGo will not process card payments; all payments are cash-only at the garage. PayHere integration and webhook handling are excluded from MVP.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
UC	Use Case
SOS	Emergency roadside assistance request triggered by customer breakdown
JWT	JSON Web Token — signed token format for stateless authentication
OTP	One-Time Password — time-limited SMS verification code
ETA	Estimated Time of Arrival — projected mechanic arrival at breakdown location
2W	Two-wheeler vehicle (motorcycle, scooter, three-wheeler)
4W	Four-wheeler vehicle (car, van, SUV, jeep)
NIC	National Identity Card — Sri Lankan government-issued identification number
GPS	Global Positioning System — satellite-based location tracking
API	Application Programming Interface — HTTP endpoints for client-server communication
Webhook	HTTP POST callback initiated by external service to notify GarageGo of events
BR	Business Rule — constraint or policy governing system behaviour

Term	Definition
MVP	Minimum Viable Product — the smallest feature set delivering core value
Socket.io	Real-time bidirectional event-based communication library
Redis	In-memory data structure store used for caching and session management
CI/CD	Continuous Integration / Continuous Deployment — automated build and release pipeline
SLA	Service Level Agreement — measurable performance commitment

1.4 References

SENG 31242 Course Module Guide — System Design Project - University of Kelaniya, 2026 SENG 31242 System Design Project Guideline & Industrial Standards - University of Kelaniya, 2026 Customer Interview Findings (Issue #9) - research/fact-finding/interviews/findings-summary.md

---

1.5 Overview

The remainder of this document is organised into five major sections. Section 2 describes the overall product perspective, defines three primary user personas, and establishes the operating environment, design constraints, and system dependencies. Section 3 details system features through formal use case descriptions, use case diagrams, activity diagrams, and system sequence diagrams. Section 4 enumerates functional requirements (FR-01 to FR-24+) with priority levels, source traceability, and verification methods. Section 5 specifies non-functional requirements across six categories: Performance, Security, Usability, Reliability, Scalability, and Maintainability. Section 6 presents alternative solutions analysis, a four-dimensional feasibility study (Technical, Economic, Operational, Schedule), and the justified recommendation for GarageGo's chosen architecture.

2. Overall Description

2.1 Product Perspective

GarageGo is a new, standalone software product. It does not replace an existing automated system but digitises a currently manual process involving phone calls, WhatsApp messages and physical visits.

The system operates as a multi-sided platform with four primary client applications and one administrative interface:

- **Customer Mobile App:** Discovery, booking, emergency SOS and review
- **Garage Owner Mobile App:** Profile management, booking handling, mechanic dispatch and invoicing
- **Mechanic Mobile App:** Job reception, GPS tracking, status updates and navigation
- **Admin Web Dashboard:** Oversight, garage approval, complaint resolution and analytics

---

2.2 User Classes and Characteristics

Persona 1: Kasun — Customer

- **Demographics:** 28 years old, software developer, residing in Colombo
- **Vehicle:** Toyota Corolla 2018 (4W), also owns a Honda Dio scooter (2W)
- **Tech Profile:** Tech-savvy daily smartphone user. active on PickMe, UberEats, and online banking apps
- **Goals:** Find reliable garages quickly, avoid overcharging, maintain digital service history, get emergency help at night
- **Pain Points:** Price opacity ("They always say one thing and then add extra parts"), quality uncertainty, emergency vulnerability
- **Frequency:** 2–3 garage visits per year (routine service + emergency)
- **Quote (from Interview #9):** *"I want to see the price before they start working. And I want to know they are actually good—not just cheap."*

Persona 2: Nimal — Garage Owner

- **Demographics:** 45 years old, owns and operates a 5-bay garage in Jaffna
- **Business:** Family-run garage established 2010; 3 full-time mechanics; services both two-wheel and four-wheel vehicles

- **Tech Profile:** Moderate smartphone user. currently uses WhatsApp Business for informal bookings and a paper logbook
- **Goals:** Fill empty service bays, reduce no-shows, attract new customers beyond word-of-mouth, manage mechanic schedules digitally
- **Pain Points:** No-shows waste 2–3 hours per day, no digital presence means losing customers to competitors, cash handling creates security risks
- **Frequency:** 8–12 customers on weekdays, 4–6 on weekends
- **Needs:** Real-time capacity dashboard, verified customer base to reduce no-shows, digital payment reconciliation with automatic commission deduction

Persona 3: Jothini — Administrator

- **Demographics:** 23 years old, platform operations manager employed by GarageGo
- **Role:** Full-time staff. not affiliated with any garage; reports to founding team
- **Tech Profile:** Advanced user. manages multiple dashboards and analytics tools. comfortable with SQL and data exports
- **Goals:** Maintain platform quality and trust, resolve disputes fairly, onboard garages efficiently, monitor for fraud patterns
- **Pain Points:** Manual verification of garage business registration documents is time-consuming, dispute evidence is scattered across chat logs and emails
- **Frequency:** Daily platform monitoring (2 hours), weekly report generation, ad-hoc complaint resolution
- **Needs:** Automated approval workflows with document checklist, structured complaint escalation paths, analytics exports for investor reporting

---

2.3 Operating Environment

Component	Requirement
Customer Mobile App	iOS 15.0+ or Android 10.0+ (API level 29+). GPS-enabled. 4G/LTE or WiFi
Garage Owner Mobile App	iOS 15.0+ or Android 10.0+. camera for document upload. 4G/LTE or WiFi
Mechanic Mobile App	iOS 15.0+ or Android 10.0+. GPS always-on during dispatch. 4G/LTE minimum
Admin Web Dashboard	Chrome 110+, Firefox 108+, Safari 16+. 1920×1080 resolution recommended
Backend Server	Node.js 20 LTS, PostgreSQL 15, Redis 7, Docker containers
Network	4G/LTE or WiFi required for real-time features. offline mode limited to cached garage lists
Geographic	Sri Lanka only. GPS accuracy dependent on local cell tower density (3–10m urban, 10–50m rural)

2.4 Design and Implementation Constraints

1. **Cash-Only Payment Model:** GarageGo will not integrate online payment gateways. All transactions are settled in cash directly between the customer and the garage. The platform records payment status manually via garage owner confirmation. This eliminates payment gateway fees, chargeback risks, and PCI-DSS compliance requirements but requires trust-based reconciliation.
2. **Map Provider Dependency:** A mapping service is required for address geocoding, distance calculations, and turn-by-turn navigation. No viable local alternative provides equivalent coverage for Sri Lankan road networks.
3. **Cross-Platform Development:** A cross-platform mobile development framework is required to maintain a single codebase for iOS and Android. Native module usage is limited to framework capabilities.

4. **API Budget Constraints:** Third-party API usage (maps, notifications) may have free tier limits. System design must respect these limits or implement usage quotas.
5. **Data Residency:** User personal data (NIC, phone numbers, location history) must remain within jurisdiction-compliant infrastructure. Regional or local cloud hosting is preferred.

2.5 Assumptions and Dependencies

1. **Smartphone Penetration:** All garage owners and mechanics possess smartphones with active data plans. This will be validated in garage owner interviews.
2. **GPS Availability:** Emergency SOS assumes GPS signal is available at the customer's location. Degraded accuracy (e.g., indoors, dense urban canyons) is handled via manual location pin confirmation.
3. **Network Connectivity:** Real-time features assume intermittent but present mobile data coverage. No offline-first architecture is planned for MVP; basic caching supports garage list browsing only.
4. **Garage Cooperation:** Garages will maintain accurate capacity counts and working hours in the app. Inaccurate data may lead to booking conflicts handled via cancellation policies.
5. **Cash Payment Trust:** Customers carry sufficient cash for garage services. Garage owners manually confirm cash receipt in the app within 24 hours of service completion. No platform-held escrow, digital wallet, or deferred payment mechanism is provided in MVP.
6. **External API Stability:** Third-party service providers (mapping, notifications, SMS) maintain their current API contracts, pricing, and service levels during the development period.
7. **User Authentication:** All users complete phone OTP verification during registration. Email verification is optional for customers but mandatory for garage owners.

---

3. System Analysis and Use-Case Modelling

3.1 Fact-Finding Techniques

The requirements for GarageGo were gathered using two formal fact-finding techniques: structured interviews and a questionnaire survey. These techniques provided both qualitative and quantitative evidence to support the identification and validation of system requirements.

3.1.1 Structured Interviews

Three vehicle owners were interviewed using a predefined interview guide to understand challenges related to finding garages, obtaining repair services, and handling emergency vehicle breakdown situations.

Evidence:

- Interview Guide
- Participant 1 Summary
- Participant 2 Summary
- Participant 3 Summary
- Interview Findings Summary

GarageGo – Customer Interview Guide

Project
GarageGo – Smart Garage & Emergency Repair Platform

Purpose
This interview is conducted to understand the challenges faced by Sri Lankan vehicle owners when finding garages and emergency vehicle repair services. The information collected will help improve the GarageGo system requirements and features.

Introduction Script
"Hello, thank you for participating in this interview.
We are conducting this interview for our university project called GarageGo – Smart Garage & Emergency Repair Platform.
The purpose of this discussion is to understand your experiences with vehicle repairs, garages, and emergency breakdown situations. Your answers will only be used for academic purposes, and your personal information will remain anonymous.
Do we have your verbal consent to continue with this interview?"
☐ Yes
☐ No

Section A – Current Behaviour

Q1. What type of vehicle do you use?

- Motorcycle
- Car
- Van
- Three-wheeler
- Other

Q2. How do you usually find a garage when your vehicle needs repair or service?

Q3. How long does it usually take to find and book a garage service?

Q4. Have you ever experienced a vehicle breakdown during an emergency situation?

- Yes
- No

If yes:

- What happened?
- Where did it happen?
- How did you get help?

Q5. How long did you wait to receive help during the breakdown?

- Less than 30 minutes
- 30 minutes – 1 hour
- 1–2 hours
- More than 2 hours

Section B – Pain Points

Q6. What is the most frustrating part of finding a garage or mechanic?

Q7. What improvements would you like to see in vehicle repair services?

Section C – App Willingness & Features

Q8. Would you feel comfortable using a mobile app to find nearby garages and mechanics?

- Yes
- No
- Maybe

Please explain why.

Q9. Which features would be most useful for you in an app like GarageGo?

Possible features:

- Emergency SOS request
- Live mechanic tracking
- Online booking
- Transparent pricing
- Ratings and reviews
- Service history

Closing Statement

"Thank you very much for your time and valuable feedback.
Your responses will help us better understand user needs and improve the GarageGo platform.
All responses will remain anonymous and used only for academic purposes."

Interview Notes Section

Topic	**Notes**
Main Pain Points	
Requested Features	
Important Quotes	
Additional Comments	

Participant1 Summary

Participant Information

Field	**Details**
Participant Code	**P-01**
Vehicle Type	**Motorcycle**
Date	**11 May 2026**
Interview Method	**In-person**
Consent Obtained	**Yes (Verbal Consent)**

Answers

Q1. What type of vehicle do you use?

Motorcycle

Q2. How do you usually find a garage when your vehicle needs repair or service?
I usually ask friends or nearby people for recommendations.

Q3. How long does it usually take to find and book a garage service?
Around 30 minutes to 1 hour.

Q4. Have you ever experienced a vehicle breakdown during an emergency situation?
Yes. My motorcycle stopped while returning home at night.
What happened?
The engine suddenly stopped working.
Where did it happen?
Near a main road in Jaffna.
How did you get help?
I contacted a nearby mechanic through a friend.

Q5. How long did you wait to receive help during the breakdown?
About 1 hour.

Q6. What is the most frustrating part of finding a garage or mechanic?
It is difficult to find trusted mechanics quickly during emergencies.

Q7. What improvements would you like to see in vehicle repair services?
Faster emergency support and better communication.

Q8. Would you feel comfortable using a mobile app to find nearby garages and mechanics?
Yes.
Explanation
It would save time and make finding help easier.

Q9. Which features would be most useful for you in an app like GarageGo?

- **Emergency SOS request**
- **Live mechanic tracking**
- **Ratings and reviews**

Important Quotes
“During emergencies, it is hard to know who to contact quickly.”
“Tracking the mechanic would make the service more reliable.”

---

3 Key Takeaways

1. **Emergency support should be faster.**
2. **Users need trusted mechanics and garages.**
3. **Live tracking is an important feature for users.**

Participant2 Summary

Participant Information

Field	Details
Participant Code	P-02
Vehicle Type	Motorcycle
Date	11 May 2026
Interview Method	Phone Interview
Consent Obtained	Yes (Verbal Consent)

Answers
Q1. What type of vehicle do you use?
Motorcycle

Q2. How do you usually find a garage when your vehicle needs repair or service?
I ask nearby shops.

Q3. How long does it usually take to find and book a garage service?
Usually 20 to 30 minutes.

Q4. Have you ever experienced a vehicle breakdown during an emergency situation?
Yes. My tire got damaged while travelling.
What happened?
The rear tire punctured suddenly.
Where did it happen?
On the roadside while travelling to work.
How did you get help?
I searched for a nearby repair shop.

Q5. How long did you wait to receive help during the breakdown?
Around 45 minutes.

Q6. What is the most frustrating part of finding a garage or mechanic?
Sometimes garages are unavailable, and it is hard to know which places are open.

Q7. What improvements would you like to see in vehicle repair services?
A system to quickly find nearby available garages.

Q8. Would you feel comfortable using a mobile app to find nearby garages and mechanics?
Yes.
Explanation
Most people use smartphones, so it would be convenient.

Q9. Which features would be most useful for you in an app like GarageGo?

- Nearby garage search
- Emergency repair request
- Ratings and reviews

Important Quotes
"Sometimes I waste time searching for garages that are closed."
"One app showing nearby garages would be very useful."

---

3 Key Takeaways

1. Users need real-time garage availability information.
2. Finding nearby garages quickly is important.
3. Ratings and reviews help users trust services.

Participant3 Summary

Participant Information

Field	Details
Participant Code	P-03
Vehicle Type	Car
Date	11 May 2026
Interview Method	Online Interview
Consent Obtained	Yes (Verbal Consent)

Answers
Q1. What type of vehicle do you use?
Car

Q2. How do you usually find a garage when your vehicle needs repair or service?
I usually search online or contact garages recommended by friends.

Q3. How long does it usually take to find and book a garage service?
Usually around 1 hour depending on garage availability.

Q4. Have you ever experienced a vehicle breakdown during an emergency situation?
Yes. My car had an engine issue while travelling.
What happened?
The vehicle suddenly stopped and could not continue driving.
Where did it happen?
Near a highway area while travelling with family.
How did you get help?
I contacted a nearby garage and waited for assistance.

Q5. How long did you wait to receive help during the breakdown?
Nearly 2 hours.

Q6. What is the most frustrating part of finding a garage or mechanic?
It is difficult to find available garages quickly and get confirmed booking times.

Q7. What improvements would you like to see in vehicle repair services?
A system where customers can easily book garage services and know service availability in advance.

Q8. Would you feel comfortable using a mobile app to find nearby garages and mechanics?
Yes.
Explanation
It would make booking services easier and save time.

Q9. Which features would be most useful for you in an app like GarageGo?

- Online booking
- Garage availability status
- Ratings and reviews
- Service history

Important Quotes
“Sometimes I call many garages before finding one that is available.”
“An online booking system would make vehicle servicing easier.”

---

3 Key Takeaways

1. Users need a faster way to book garage services.
2. Garage availability information is important for customers.
3. Online booking and reviews increase convenience and trust.

Interview Findings Summary

Findings Summary – Customer Interviews
Project
GarageGo – Smart Garage & Emergency Repair Platform
Source
Structured Customer Interviews (P-01, P-02, P-03)
Date
11 May 2026

---

1. Overview

Three structured interviews were conducted with Sri Lankan vehicle owners, including two motorcycle users and one car user. The purpose was to understand real-world problems faced when finding garages and handling emergency vehicle breakdown situations.

The findings clearly show that users face delays, lack of trusted information, and poor emergency support when dealing with vehicle repairs.

---

2. Common Themes

From all interviews, the following common patterns were identified:

- Users depend on friends or Google to find garages
- Emergency breakdowns cause high stress and delays
- Difficulty in finding available and trusted garages quickly
- Lack of real-time support during vehicle failure
- Users prefer a single platform for all garage-related needs

---

3. Top 5 Pain Points

1. Difficulty finding a nearby available garage quickly
2. Long waiting time during emergency breakdowns
3. Lack of trust in unknown garages or mechanics
4. No real-time tracking or status updates of service
5. No clear system for booking or confirming garage availability

---

4. Feature Requests

Users suggested the following key features for improvement:

- Emergency SOS request system for breakdown situations
- Live tracking of mechanic/assistance arrival
- Online garage booking system with availability status
- Ratings and reviews for trust building
- Nearby garage search with real-time availability

---

5. Pain Point → Functional Requirement Mapping

Pain Point	Functional Requirement
Difficulty finding nearby garage	System shall allow users to search nearby garages using location
Long emergency waiting time	System shall support emergency SOS request and dispatch
Lack of trust in garages	System shall provide ratings and reviews for garages
No real-time updates	System shall provide live tracking of mechanic/service status
No booking system	System shall allow users to book garage services online

6. Conclusion

The interviews confirm a strong need for a unified platform like GarageGo. The main requirement is to reduce delays during emergencies and improve trust through real-time information, booking, and tracking features.
These findings directly support the core functional requirements of the GarageGo system.

3.1.2 Questionnaire Survey

A questionnaire survey was distributed among Sri Lankan vehicle owners to collect quantitative data regarding vehicle servicing habits, garage-finding challenges, emergency breakdown experiences, and desired application features.

Evidence:

- Survey Design
- Questionnaire Results Summary
- Questionnaire Key Findings

GarageGo – Survey Design

Project

GarageGo – Smart Garage & Emergency Repair Platform

Purpose

This survey is conducted to understand the challenges faced by Sri Lankan vehicle owners when finding garages and emergency repair services. The responses collected will support the requirements engineering process of the GarageGo system and help identify user needs, pain points, and expected features.

Survey Questionnaire

Section A – Respondent Profile

Q1. What type of vehicle do you use?

Question Type: Multiple Choice

- Car
- Motorcycle
- Van
- Three-wheeler
- Other

Q2. How often do you need vehicle service/repair?

Question Type: Multiple Choice

- Monthly
- Every 3–6 months
- Once a year
- Rarely

Q3. How do you currently find a garage?

Question Type: Multiple Choice

- Friends or family suggestion
- Go and find nearby garage
- My regular mechanic
- Google Search
- Other

Section B – Current Problems

Q4. It is difficult to find an available garage quickly.

Question Type: Linear Scale (1–5)

- 1 = Strongly Disagree
- 5 = Strongly Agree

Q5. I experience long waiting times at garages.

Question Type: Linear Scale (1–5)

- 1 = Strongly Disagree
- 5 = Strongly Agree

Q6. I have faced unfair or unclear pricing.

Question Type: Linear Scale (1–5)

- 1 = Strongly Disagree
- 5 = Strongly Agree

Q7. I find it difficult to trust unknown garages/mechanics.
Question Type: Linear Scale (1–5)

- 1 = Strongly Disagree
- 5 = Strongly Agree

Section C – Emergency Experience
Q8. Have you ever faced a vehicle breakdown in an emergency?
Question Type: Multiple Choice

- Yes
- No

Q9. If yes, how long did you usually wait for help?
Question Type: Multiple Choice

- 30–60 minutes
- 1–2 hours
- More than 2 hours
- Never got help quickly

Q10. Would you use a service that sends a mechanic within 30 minutes?
Question Type: Multiple Choice

- Yes
- No
- Maybe

Section D – Feature Preferences
Q11. Which features are most important to you? (Select up to 3)
Question Type: Checkbox

- Emergency service
- Live tracking of mechanic
- Advance garage booking
- Transparent pricing
- Ratings & reviews of garages

Section E – App Acceptance
Q12. Would you use a mobile app like GarageGo?
Question Type: Multiple Choice

- Yes
- No
- Maybe

Q13. What do you expect from this app?
Question Type: Short Answer

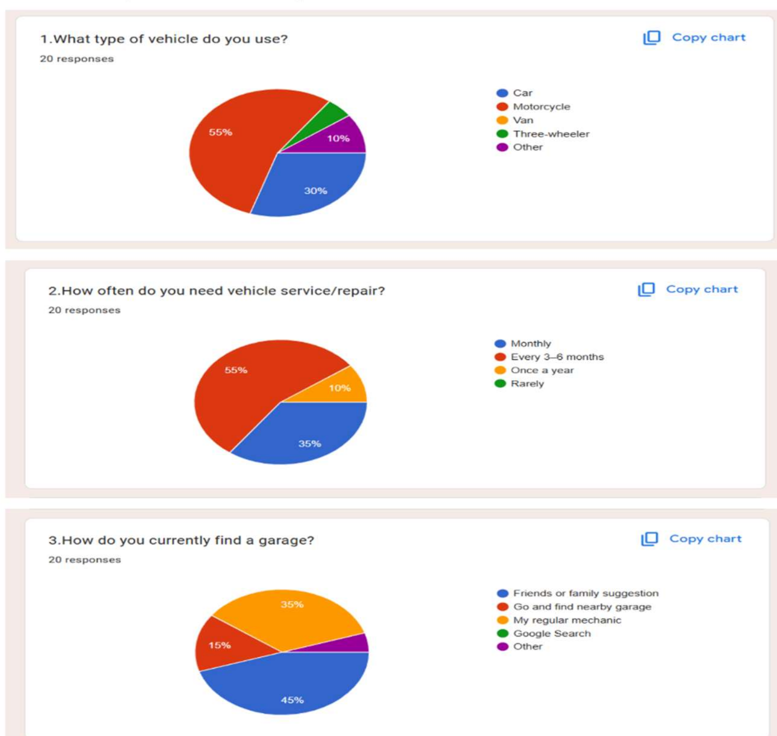
Survey Distribution
The survey was distributed among Sri Lankan vehicle owners using online platforms such as WhatsApp and social media groups. Responses were collected anonymously for academic and research purposes only.

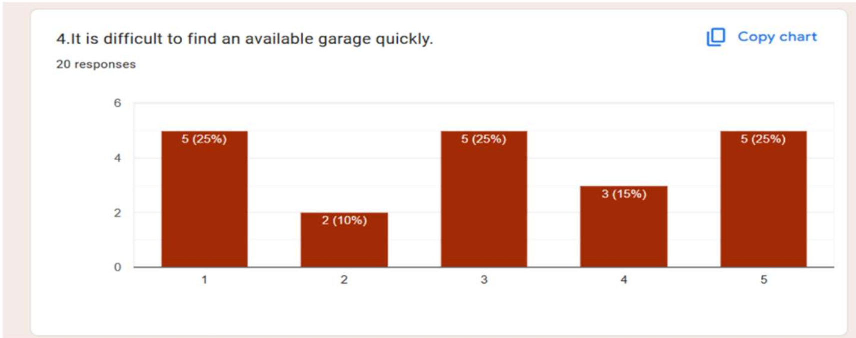
Expected Outcome
The survey findings will help:

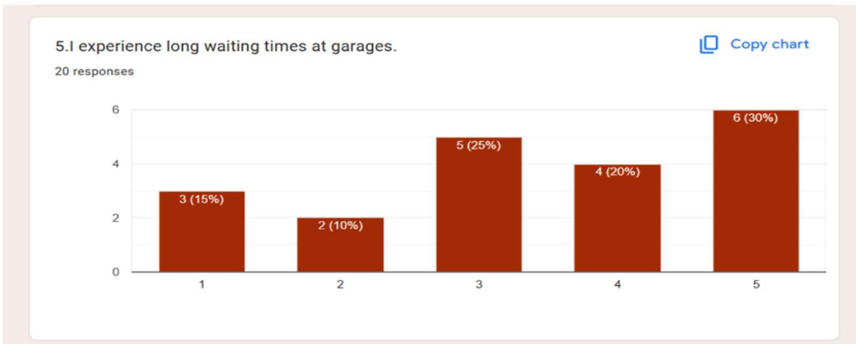
- Identify common vehicle repair challenges
- Understand emergency breakdown experiences
- Validate user interest in GarageGo
- Prioritize important application features
- Support functional requirement creation in the SRS

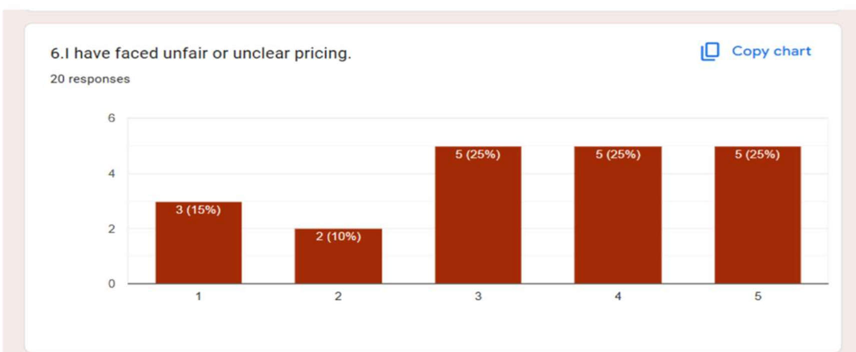
<u>GarageGo – Results Summary</u>

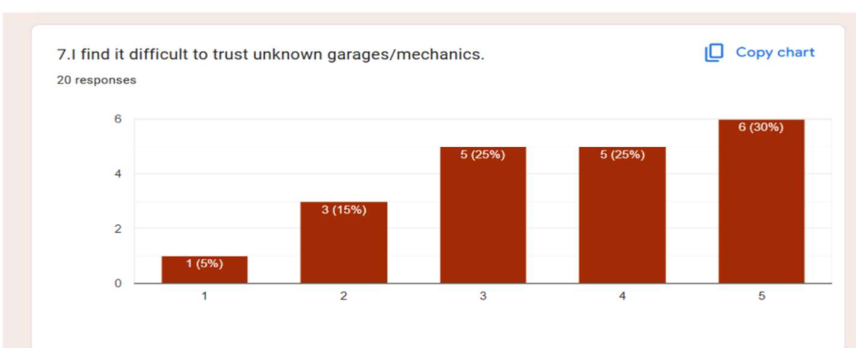
Results Summary from Customer Survey

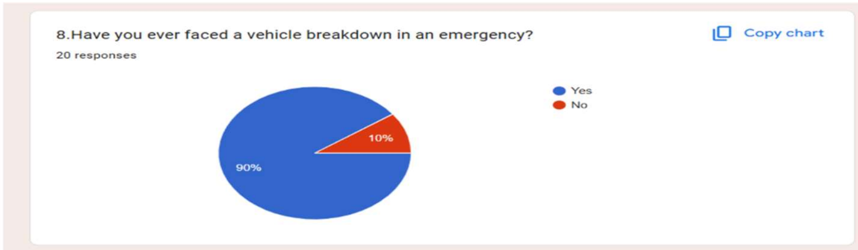



4.It is difficult to find an available garage quickly.
20 responses
Copy chart
5 (25%)
2 (10%)
5 (25%)
3 (15%)
5 (25%)

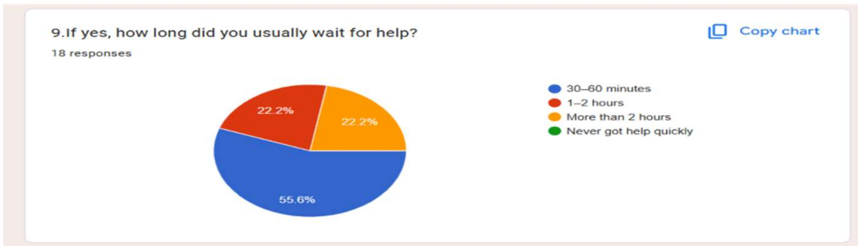

5.I experience long waiting times at garages.
20 responses
Copy chart
3 (15%)
2 (10%)
5 (25%)
4 (20%)
6 (30%)


6.I have faced unfair or unclear pricing.
20 responses
Copy chart
3 (15%)
2 (10%)
5 (25%)
5 (25%)
5 (25%)

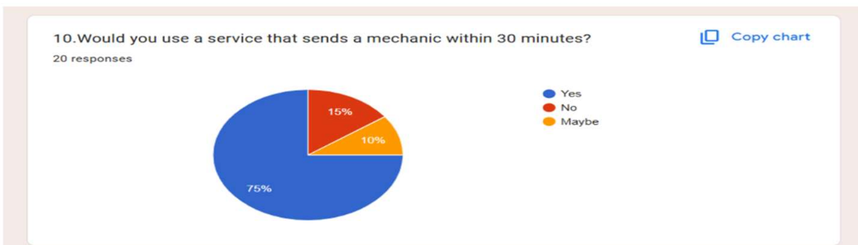

7.I find it difficult to trust unknown garages/mechanics.
20 responses
Copy chart
1 (5%)
3 (15%)
5 (25%)
5 (25%)
6 (30%)



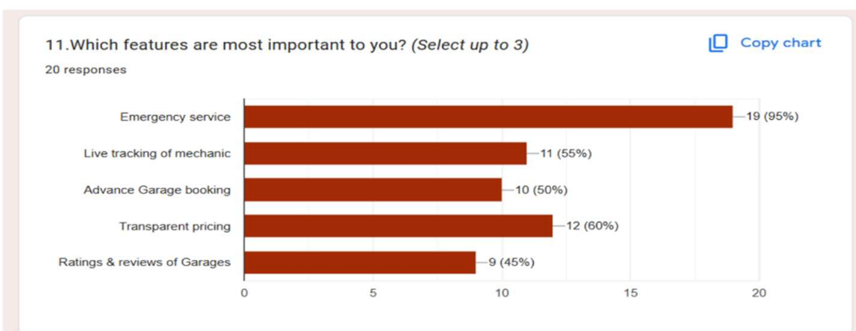
8.Have you ever faced a vehicle breakdown in an emergency?
20 responses
Copy chart
Yes
No
10%
90%



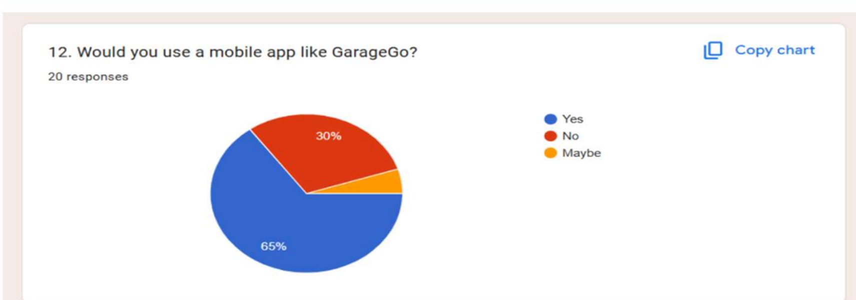
9.If yes, how long did you usually wait for help?
18 responses
Copy chart
30–60 minutes
1–2 hours
More than 2 hours
Never got help quickly
22.2%
22.2%
55.6%



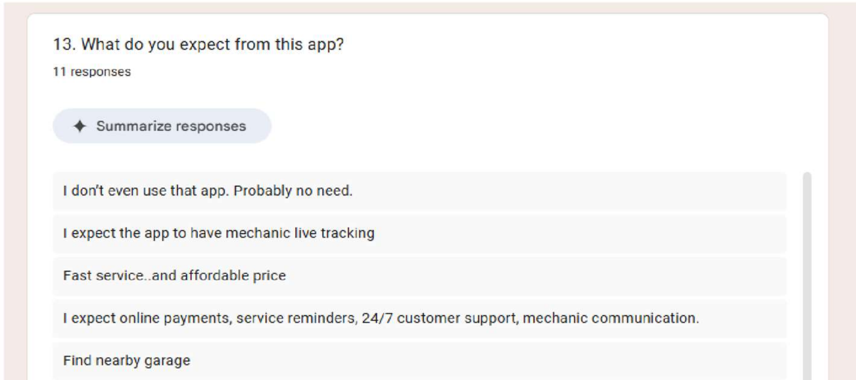
10.Would you use a service that sends a mechanic within 30 minutes?
20 responses
Copy chart
Yes
No
Maybe
15%
10%
75%



11.Which features are most important to you? (Select up to 3)
20 responses
Copy chart
Emergency service
19 (95%)
Live tracking of mechanic
11 (55%)
Advance Garage booking
10 (50%)
Transparent pricing
12 (60%)
Ratings & reviews of Garages
9 (45%)
0
5
10
15
20



12. Would you use a mobile app like GarageGo?
20 responses
Copy chart
Yes
No
Maybe
30%
65%



GarageGo – Questionnaire Key Findings

Project
GarageGo – Smart Garage & Emergency Repair Platform

Overview
This document summarizes the key findings extracted from the questionnaire survey conducted among Sri Lankan vehicle owners. The goal is to identify major user problems, behaviour patterns, and feature expectations related to garage finding and emergency vehicle repair services.

Key Findings
1. Difficulty in Finding Garages
A significant number of respondents reported that it is difficult to quickly find an available garage, especially during urgent situations or breakdowns.

---

2. Long Waiting Times
Many users experience long waiting times when seeking vehicle repair services, often leading to inconvenience and delays in daily activities.

---

3. Trust Issues with Unknown Garages
Respondents indicated low trust in unfamiliar garages or mechanics, preferring known or previously used service providers.

---

4. Emergency Breakdown Situations Are Common
A considerable portion of users have experienced vehicle breakdowns, and most reported delays in receiving help during such emergencies.

---

5. Strong Demand for Emergency Assistance
Users showed strong interest in a system that can provide quick emergency roadside assistance or connect them to nearby mechanics.

---

6. Preference for Live Tracking and Transparency
Features such as live mechanic tracking, transparent service information, and reliable ratings/reviews were highly preferred by respondents.

---

7. High Acceptance of a Mobile Solution
Most respondents indicated that they would use a mobile application like GarageGo to solve garage-finding and emergency service problems.

Summary
The survey clearly indicates a strong need for a centralized platform that improves:

- Speed of finding garages
- Emergency response efficiency
- Trust and transparency in services
- Accessibility of repair support

These findings directly support the core objectives of the GarageGo system.

3.2 Existing System Analysis

Currently, vehicle owners in Sri Lanka mainly rely on manual methods to locate garages, mechanics, and roadside assistance services. Customers usually contact garages through phone calls, Facebook pages, WhatsApp messages, or personal recommendations. During emergencies such as vehicle breakdowns, users often struggle to quickly locate nearby mechanics who are available at that moment. Most garages also do not maintain a centralized booking or tracking system, causing delays, communication failures, and customer dissatisfaction.

One major pain point identified during interviews is the absence of real-time garage availability information. Customers cannot determine whether a garage has available service slots before visiting or contacting them. This frequently results in long waiting times, unnecessary travel, and wasted fuel.

Another significant issue is the lack of emergency dispatch coordination. In roadside breakdown situations, customers must manually contact multiple garages or mechanics individually. There is no automated dispatching mechanism that identifies the nearest available mechanic and assigns the request immediately.

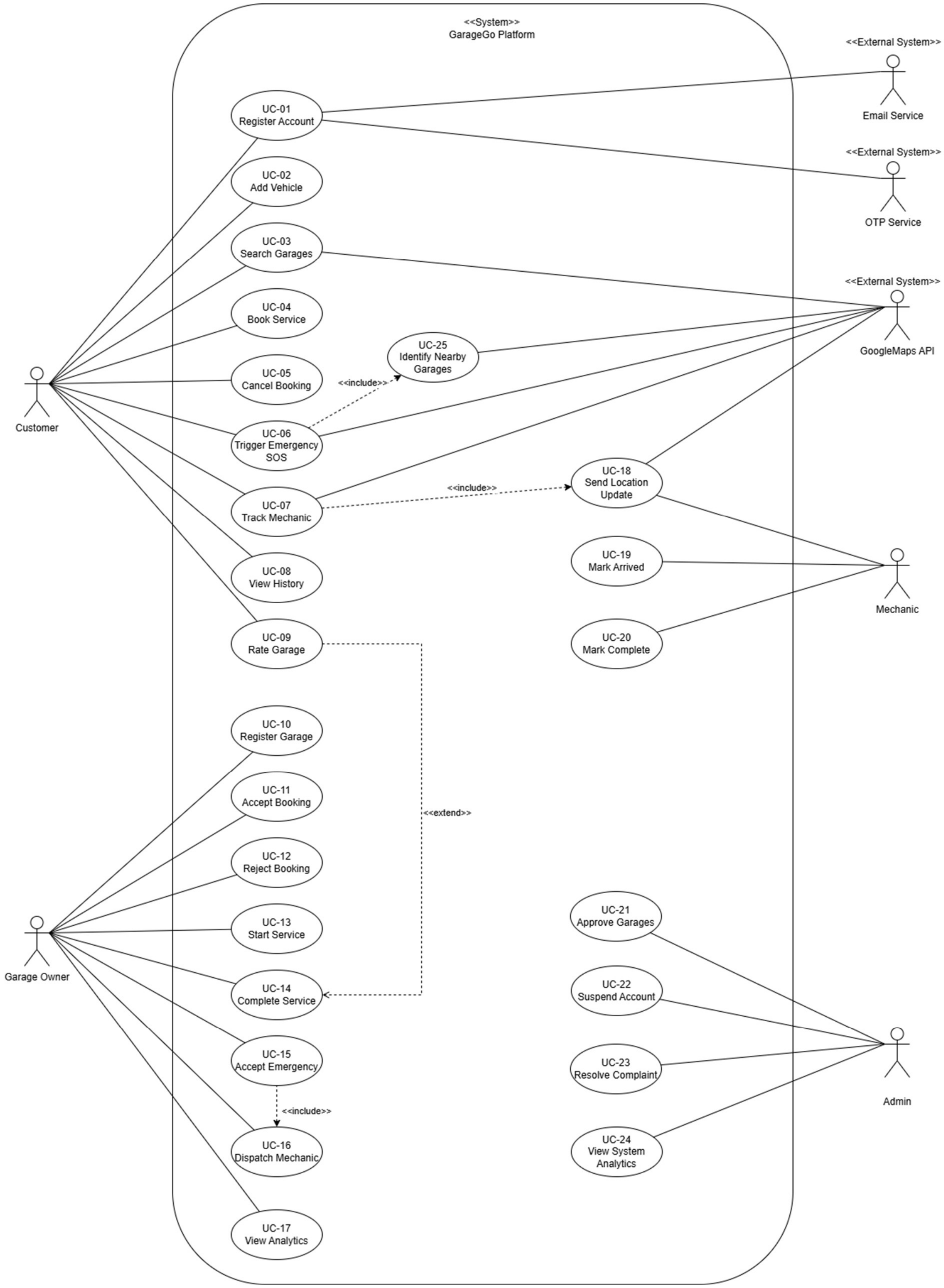
A third issue is the lack of transparent pricing and service tracking. Customers are often unaware of estimated repair costs, service progress, or repair history. This creates trust issues between vehicle owners and garages.

Additionally, many small and medium-scale garages struggle with customer management, booking organization, and service scheduling because they still rely on notebooks, paper records, and informal communication methods. GarageGo addresses these limitations by providing a centralized digital platform for garage discovery, emergency roadside assistance, booking management, and live mechanic tracking.

3.3 Use-Case Diagram

System Boundary
GarageGo Platform

Actor	**Description**
Customer	Vehicle owner using the platform
Garage Owner	Garage manager handling bookings and emergencies
Mechanic	Mechanic assigned to repair and emergency tasks
Admin	Platform administrator
Google Maps API	External mapping and GPS service
OTP Service	External SMS verification service
Email Service	External email notification service


<<System>>
GarageGo Platform
UC-01 Register Account
UC-02 Add Vehicle
UC-03 Search Garages
UC-04 Book Service
UC-05 Cancel Booking
UC-06 Trigger Emergency SOS
UC-07 Track Mechanic
UC-08 View History
UC-09 Rate Garage
UC-10 Register Garage
UC-11 Accept Booking
UC-12 Reject Booking
UC-13 Start Service
UC-14 Complete Service
UC-15 Accept Emergency
UC-16 Dispatch Mechanic
UC-17 View Analytics
UC-18 Send Location Update
UC-19 Mark Arrived
UC-20 Mark Complete
UC-21 Approve Garages
UC-22 Suspend Account
UC-23 Resolve Complaint
UC-24 View System Analytics
UC-25 Identify Nearby Garages
<<include>>
<<include>>
<<extend>>
<<include>>
Customer
Garage Owner
Mechanic
Admin
<<External System>>
Email Service
<<External System>>
OTP Service
<<External System>>
GoogleMaps API

Customer Use Cases

- Register Account
- Add Vehicle
- Search Garages
- Book Service
- Cancel Booking
- Trigger Emergency SOS
- Track Mechanic
- View Service History
- Rate Garage

Garage Owner Use Cases

- Register Garage
- Accept Booking
- Reject Booking
- Start Service
- Complete Service
- Accept Emergency
- Dispatch Mechanic
- View Analytics

Mechanic Use Cases

- Send Location Update
- Mark Arrived
- Mark Complete

Admin Use Cases

- Approve Garage
- Suspend Account
- Resolve Complaint
- View System Analytics

External System Interactions

- Google Maps API
- OTP Service
- Email Service

3.4 Use-Case Descriptions

UC-01 – Register Account

Field	Content
Use Case ID	UC-01
Use Case Name	Register Account
Actor(s)	Customer, Garage Owner, Admin, OTP Service, Email Service
Preconditions	User has installed the application and has internet access
Postconditions – Success	User account is created and verified successfully
Postconditions – Failure	Account is not created; validation or verification error displayed

Main Flow

1. User opens registration screen.
2. User enters personal details.
3. User enters phone number and email address.
4. System validates entered information.
5. System sends OTP to phone number.
6. User enters OTP.
7. System verifies OTP and email.
8. System creates account and logs user in.

Alternative Flow 1

- At step 4: If email or phone number already exists, system displays duplicate account message.

Alternative Flow 2

- At step 6: If OTP is invalid or expired, system requests a new OTP.

Business Rules

- BR-01: Phone verification is mandatory.
- BR-02: Email verification is required.
- BR-03: Password must meet security requirements.

UC-02 – Add Vehicle

Field	Content
Use Case ID	UC-02
Use Case Name	Add Vehicle
Actor(s)	Customer
Preconditions	Customer is logged into the system
Postconditions – Success	Vehicle is added to customer profile
Postconditions – Failure	Vehicle record is not saved

Main Flow

1. Customer opens vehicle management section.
2. Customer selects "Add Vehicle".
3. Customer enters vehicle details.
4. System validates registration number and required fields.
5. Customer submits form.
6. System stores vehicle information successfully.

Alternative Flow 1

- At step 4: If registration number already exists under another account, system displays error message.

Alternative Flow 2

- At step 3: Customer may optionally add notes and photos.

Business Rules

- BR-04: Each vehicle must have a unique registration number.
- BR-05: Vehicle type must match supported categories.

UC-03 – Search Garages

Field	Content
Use Case ID	UC-03
Use Case Name	Search Garages
Actor(s)	Customer, Google Maps API
Preconditions	Customer is logged in and location access is enabled
Postconditions – Success	Matching garages are displayed on map and list
Postconditions – Failure	No garages displayed or location retrieval fails

Main Flow

1. Customer opens garage search page.
2. System retrieves customer location.
3. Customer applies filters if needed.
4. System queries nearby approved garages.
5. System displays garages with ratings and availability.

Alternative Flow 1

- At step 2: If GPS permission is denied, system requests manual location entry.

Alternative Flow 2

- At step 4: If no garages are found, system displays "No nearby garages available".

Business Rules

- BR-06: Only approved garages are shown.
- BR-07: Results are sorted by distance and availability.

UC-04 – Book Service

Field	**Content**
Use Case ID	UC-04
Use Case Name	Book Service
Actor(s)	Customer, Garage Owner
Preconditions	Customer is logged in and has at least one registered vehicle
Postconditions – Success	Booking request is created and notification sent to garage owner
Postconditions – Failure	Booking is not created

Main Flow

1. Customer selects a garage.
2. Customer selects vehicle and required service.
3. Customer chooses date and time.
4. System calculates estimated cost.
5. Customer confirms booking.
6. System creates booking request and sends notification to garage owner.

Alternative Flow 1

- At step 3: If selected slot is unavailable, system requests another time slot.

Business Rules

- BR-08: Garage owner must respond within two hours.
- BR-09: Booking status initially becomes "Pending".

UC-05 – Cancel Booking

Field	Content
Use Case ID	UC-05
Use Case Name	Cancel Booking
Actor(s)	Customer
Preconditions	Customer has an active booking
Postconditions – Success	Booking is cancelled and a penalty added for the next booking
Postconditions – Failure	Booking remains active

Main Flow

1. Customer opens booking details.
2. Customer selects “Cancel Booking”.
3. System checks cancellation policy.
4. System cancels booking.
5. Customer receives cancellation confirmation.

Alternative Flow 1

- At step 3: If garage already accepted booking, cancellation fee may apply.

Alternative Flow 2

- At step 4: If booking is already in service, cancellation is rejected.

Business Rules

- BR-10: Cancellation fee applies after acceptance.
- BR-11: Outstanding balances are added to future invoices.

UC-06 – Trigger Emergency SOS

Field	Content
Use Case ID	UC-06
Use Case Name	Trigger Emergency SOS
Actor(s)	Customer, Garage Owner, Google Maps API
Preconditions	Customer is logged in and location services are enabled
Postconditions – Success	Emergency request is broadcast to nearby garages
Postconditions – Failure	Emergency request is not created

Main Flow

1. Customer presses SOS button.
2. Customer selects affected vehicle.
3. Customer enters issue description.
4. System retrieves GPS location.
5. System identifies nearby approved garages.
6. Emergency alert is sent to eligible garages.
7. First accepting garage is assigned.

Alternative Flow 1

- At step 4: If location cannot be retrieved, system asks for manual location selection.

Alternative Flow 2

- At step 7: If no garage accepts within sixty seconds, system expands search radius.

Business Rules

- BR-12: Only garages with emergency coverage receive alerts.
- BR-13: First accepted request wins assignment.

UC-07 – Track Mechanic

Field	Content
Use Case ID	UC-07
Use Case Name	Track Mechanic
Actor(s)	Customer, Mechanic, Google Maps API
Preconditions	Emergency request has been accepted
Postconditions – Success	Customer receives live mechanic location updates
Postconditions – Failure	Live tracking unavailable

Main Flow

1. Mechanic starts navigation.
2. Mechanic app sends GPS updates.
3. System updates mechanic location in real time.
4. Customer views mechanic location on map.
5. Customer receives arrival notifications.

Alternative Flow 1

- At step 2: If internet connection is lost, location updates pause temporarily.

Alternative Flow 2

- At step 4: Customer can refresh tracking manually.

Business Rules

- BR-14: GPS updates are sent every few seconds.
- BR-15: Tracking is only available for active emergencies.

UC-08 – View History

Field	Content
Use Case ID	UC-08
Use Case Name	View History
Actor(s)	Customer
Preconditions	Customer is logged in
Postconditions – Success	Service history and invoices are displayed
Postconditions – Failure	History cannot be retrieved

Main Flow

1. Customer opens history section.
2. System retrieves completed bookings.
3. System displays invoices and service details.
4. Customer selects a specific record for more details.

Alternative Flow 1

- At step 2: If no history exists, system displays empty history message.

Alternative Flow 2

- Customer may filter records by vehicle or date.

Business Rules

- BR-16: Only completed and paid services appear in history.

UC-09 – Rate Garage

Field	Content
Use Case ID	UC-09
Use Case Name	Rate Garage
Actor(s)	Customer
Preconditions	Customer completed a service
Postconditions – Success	Rating and review are saved
Postconditions – Failure	Review is not submitted

Main Flow

1. Customer opens completed booking.
2. Customer selects rating score.
3. Customer enters optional review.
4. Customer submits feedback.
5. System stores rating and updates garage score.

Alternative Flow 1

- At step 3: Customer may skip written review.

Alternative Flow 2

- If offensive language is detected, system rejects review.

Business Rules

- BR-17: Ratings range from 1 to 5 stars.
- BR-18: Only customers with completed bookings may review.

UC-10 – Register Garage

Field	**Content**
Use Case ID	UC-10
Use Case Name	Register Garage
Actor(s)	Garage Owner, Admin
Preconditions	Garage owner account exists
Postconditions – Success	Garage registration is submitted for approval
Postconditions – Failure	Garage registration is not saved

Main Flow

1. Garage owner opens registration form.
2. Owner enters business details.
3. Owner uploads required documents and photos.
4. System validates submitted data.
5. Owner submits application.
6. System marks garage status as pending approval.

Alternative Flow 1

- At step 4: If required documents are missing, system displays validation error.

Alternative Flow 2

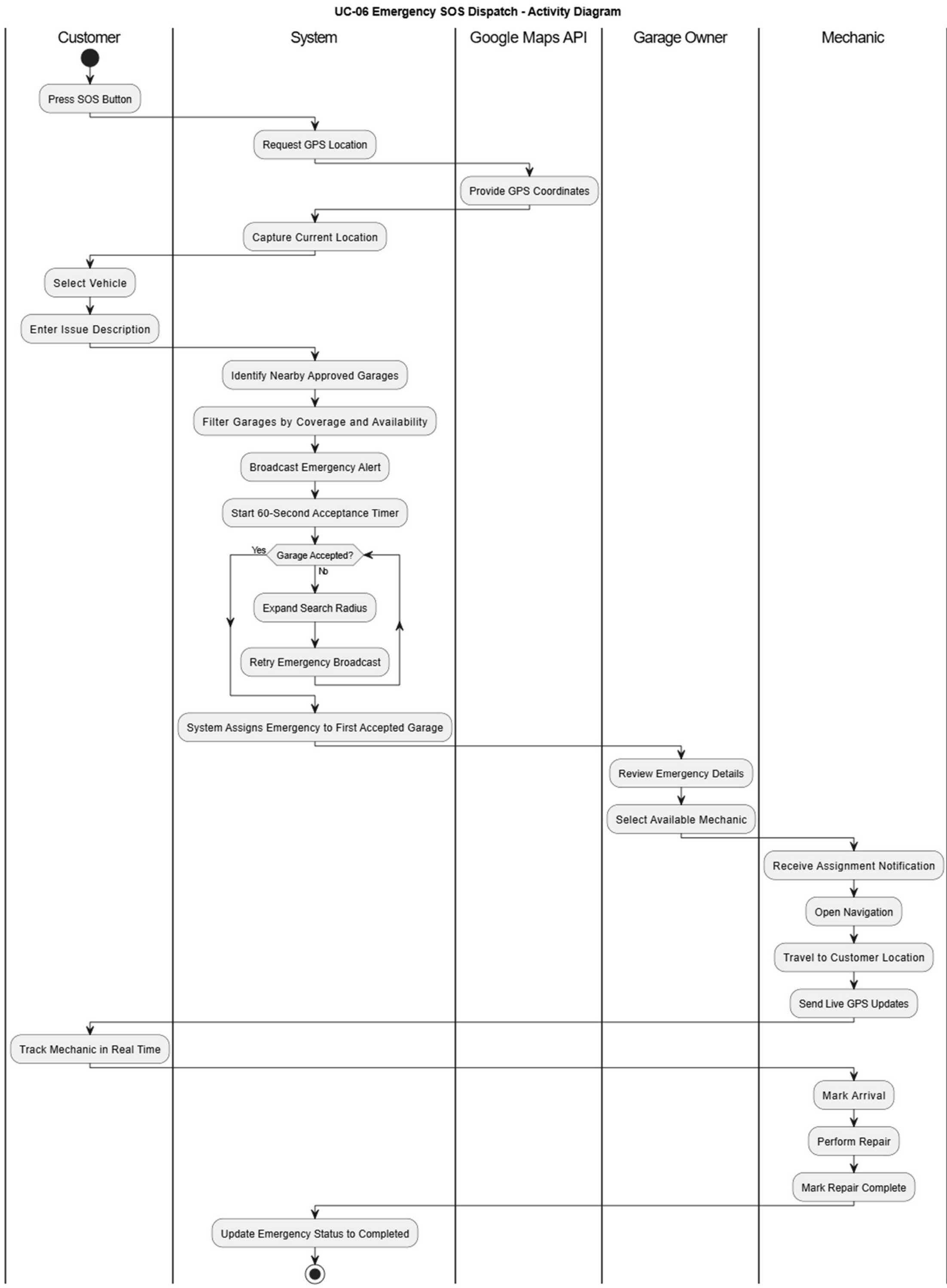
- Admin may later reject registration with a reason.

Business Rules

- BR-19: Only approved garages are visible to customers.
- BR-20: Registration number must be unique.

3.5 Activity Diagrams

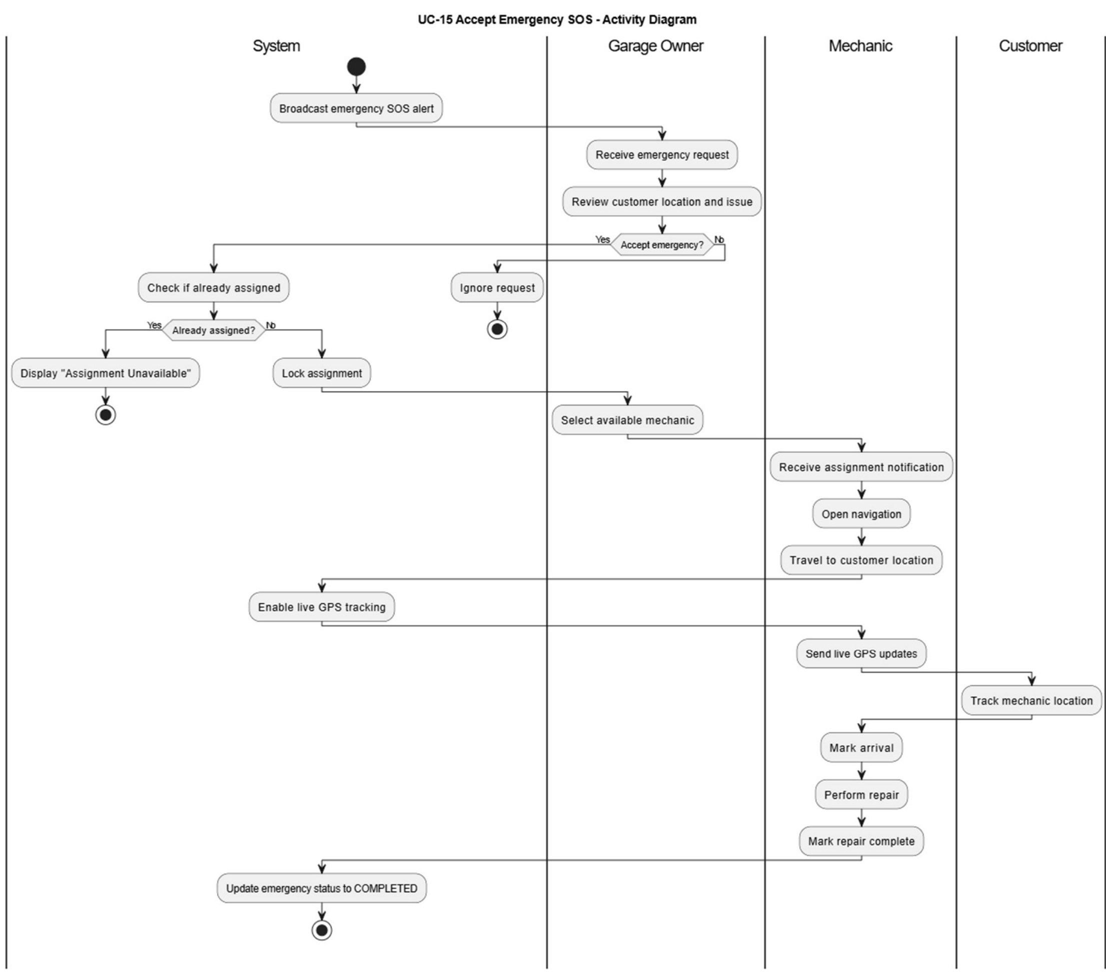
AD1 – UC-06 Emergency SOS Dispatch



Flow

- SOS Trigger
- Capture GPS
- Validate Eligibility
- Find Nearby Garages
- Broadcast Emergency Request
- Wait 60 Seconds
- First Accept Wins
- Assign Garage
- Dispatch Mechanic
- Track Mechanic
- Complete Emergency

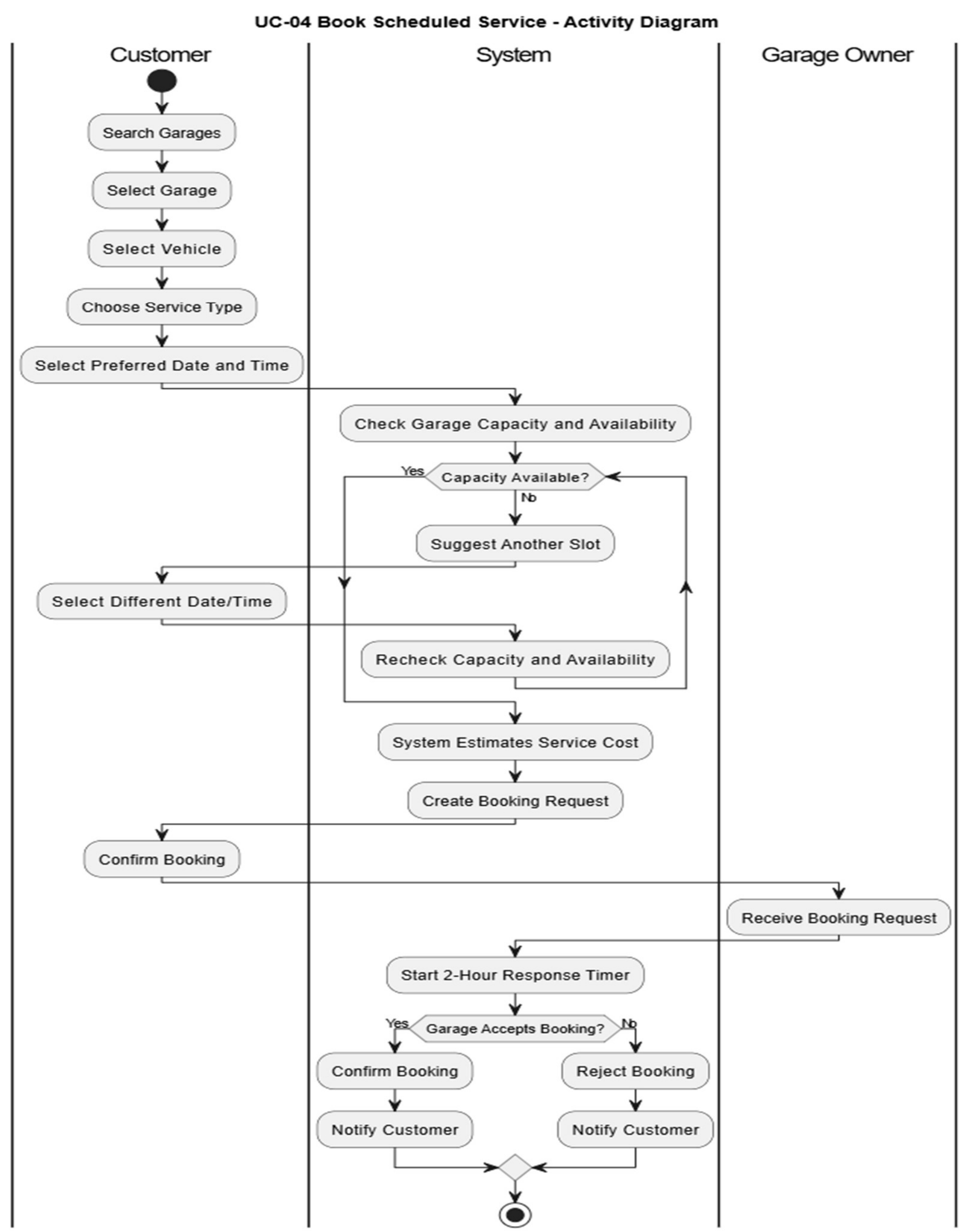
AD2 – UC-15 Accept Emergency SOS



Flow

- Receive Emergency Alert
- Review Request
- Accept Request
- Lock Assignment
- Allocate Mechanic
- Navigate to Customer
- Update Live Location
- Arrive at Customer Location
- Complete Repair

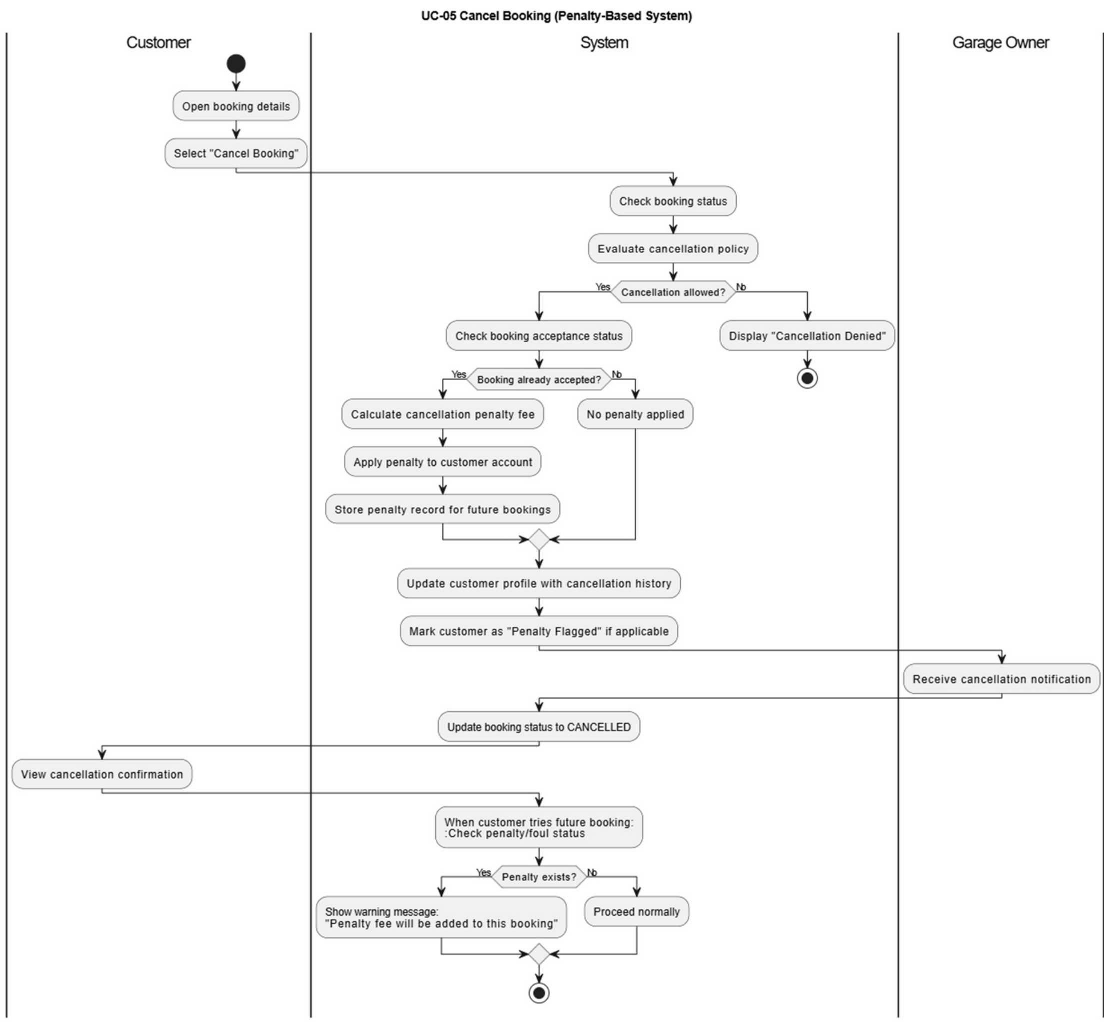
AD3 – UC-04 Book Service



Flow

- Search Garage
- Select Garage
- Select Service
- Choose Time Slot
- Validate Capacity
- Create Booking
- Start 2-Hour Response Timer
- Accept or Reject Booking
- Send Confirmation

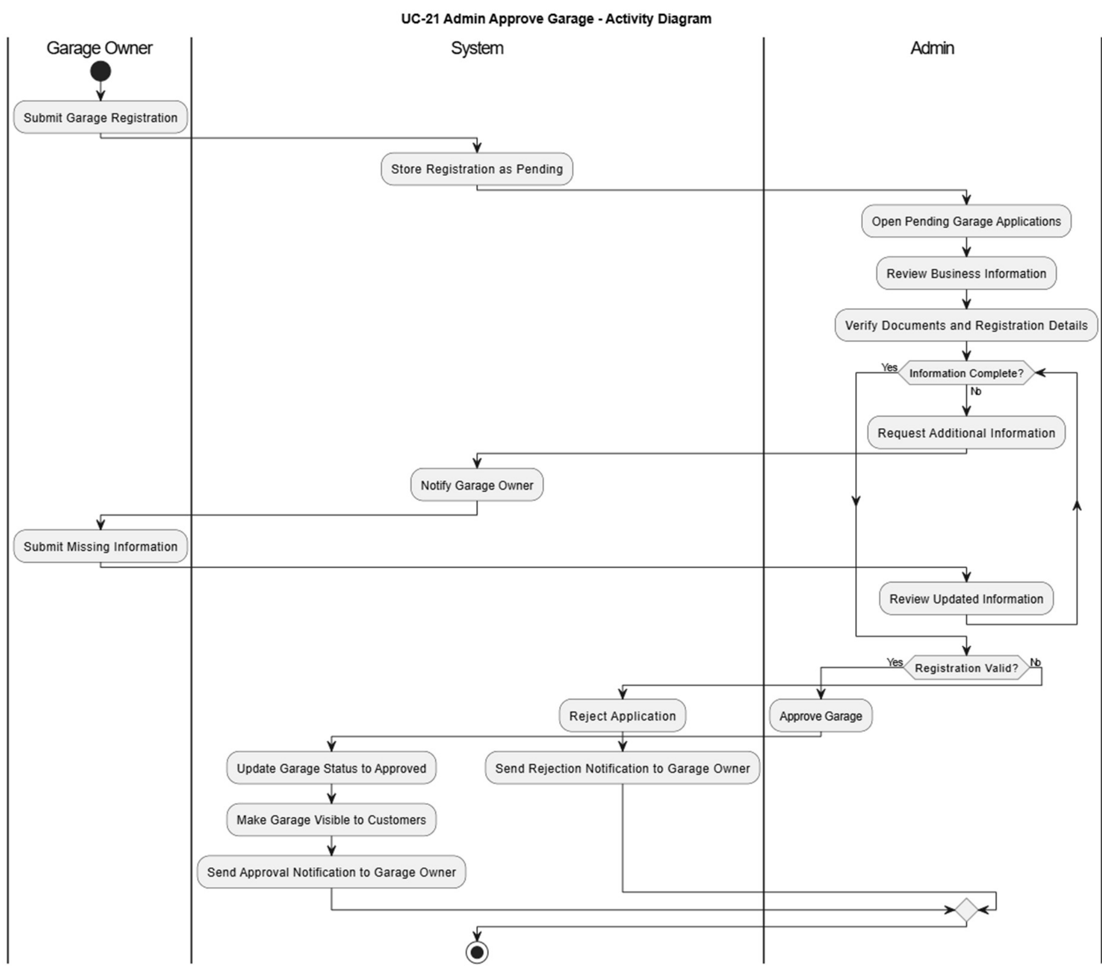
AD4 – UC-05 Cancel Booking



Flow

- Open Booking
- Validate Booking Status
- Evaluate Cancellation Policy
- Calculate Penalty
- Confirm Cancellation
- Update Booking Status
- Apply Restrictions if Required

AD5 – UC-21 Admin Garage Approval



Flow

- Receive Garage Registration
- Review Documents
- Verify Business Information
- Request Additional Information
- Approve or Reject Garage
- Notify Garage Owner

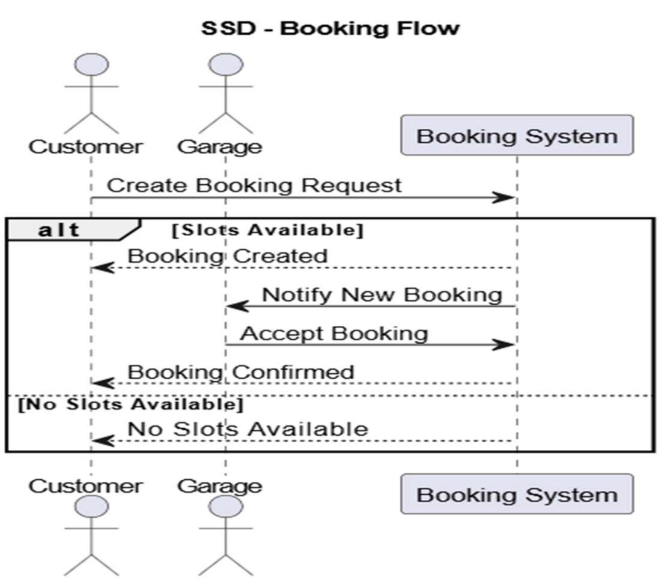
3.6 System Sequence Diagrams

This section presents the system sequence diagrams (SSDs) for the three major use cases of the GarageGo platform. Each SSD illustrates the message flows between actors and internal system components, including synchronous and asynchronous interactions, loop frames, and alternative frames where applicable. All diagrams are cross-referenced to their corresponding use-case descriptions in Section 3.4.

SSD-01 — Emergency SOS Full Flow

This diagram shows the full message flow from the moment a customer triggers an emergency SOS to the point where the mechanic's live GPS location is streamed back to the customer. It covers the API broadcast to eligible garages via Socket.io, the first-accept-wins assignment with a SELECT FOR UPDATE database lock, mechanic dispatch, and the real-time location loop.

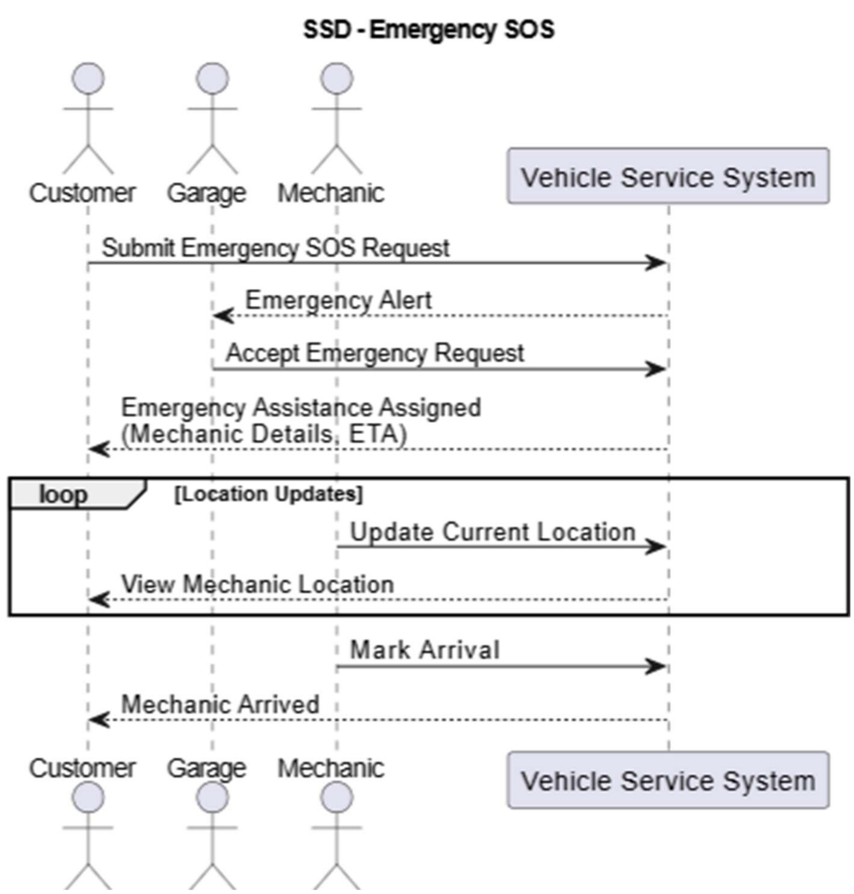
- **PNG:** documents/diagrams/sequence diagram/EmergencySOS_Final.drawio.png
- **PlantUML:** documents/diagrams/sequence-diagram/Emergency SOS.puml



SSD-02 — Booking Flow

This diagram covers the booking lifecycle from the customer's initial POST request through capacity validation, booking creation, garage notification via FCM push, and the garage owner's accept or reject response. It includes the booking state transitions (pending → accepted → in_service) and the corresponding customer notifications via Socket.io.

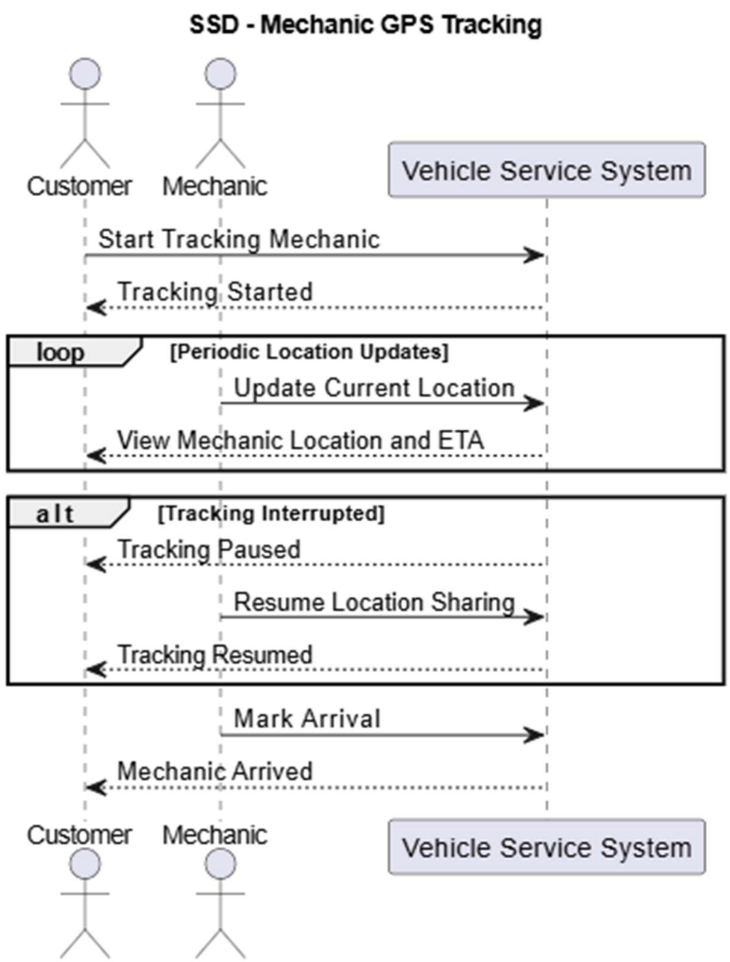
- **PNG:** documents/diagrams/sequence-diagram/BookingFlow_Final.drawio.png
- **PlantUML:** documents/diagrams/sequence-diagram/Booking Flow.puml



SSD-03 — Mechanic GPS Tracking

This diagram details the real-time GPS tracking flow during an active emergency. It includes the loop frame for mechanic location updates every 5 seconds via Socket.io and an alternative frame for the signal-loss scenario where tracking is paused after 30 seconds of no updates and resumed upon reconnection.

- **PNG:** documents/diagrams/sequence-diagram/MechanicTracking_Final.drawio.png
- **PlantUML:** documents/diagrams/sequence-diagram/Mechanic Tracking.puml



3.7 System Context Diagram

The context diagram defines the GarageGo system boundary and shows all external actors (Customer, Garage Owner/Mechanic, System Admin) and external services (Email Service, Twilio, Google Maps Platform, External Messaging System) and their interactions with the internal components (API Server, Admin Dashboard, Customer App, Garage/Mechanic App).

- **PNG:** documents/diagrams/context-diagram/Context Diagram.drawio.png
- **PlantUML:** documents/diagrams/context-diagram/context-diagram.puml

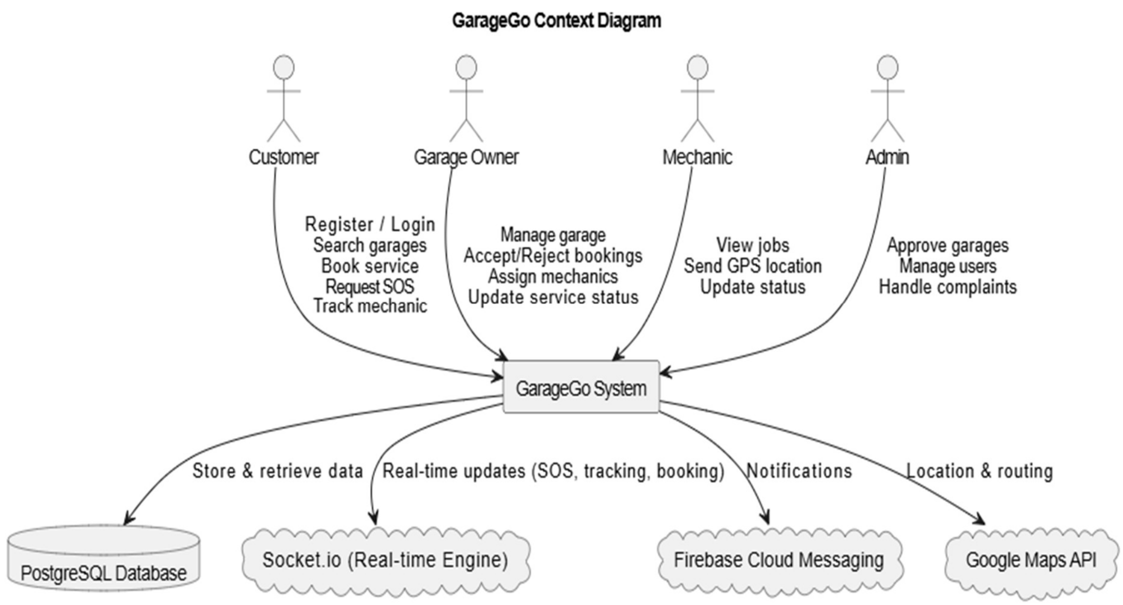


Diagram Storage Structure

```
documents/
└── diagrams/
    ├── use-case-diagram/
    │   ├── UseCaseDiagram.drawio
    │   └── UseCaseDiagram.png
    │
    ├── activity-diagrams/
    │   ├── AD_01.puml
    │   ├── AD_01.png
    │   ├── AD_02.puml
    │   ├── AD_02.png
    │   ├── AD_03.puml
    │   ├── AD_03.png
    │   ├── AD_04.puml
    │   ├── AD_04.png
    │   ├── AD_05.puml
    │   └── AD_05.png
    │
    ├── context-diagram/
    │   ├── Context Diagram.drawio.png
    │   └── context-diagram.puml
    │
    └── sequence-diagram/
        ├── Emergency SOS Sequence diagram.drawio.png
        ├── Emergency SOS.puml
        ├── Booking Flow.drawio.png
        ├── Booking Flow.puml
        ├── Mechanic Tracking GPS.drawio.png
        └── Mechanic Tracking.puml
```

Diagram Summary

Folder	Diagram	PNG File	PlantUML Source	UC Reference
use-case-diagram/	Use Case Diagram	UseCaseDiagram.png	UseCaseDiagram.drawio	All UCs
activity-diagrams/	Emergency SOS Dispatch	AD_01.png	AD_01.puml	UC-06
activity-diagrams/	Accept Emergency SOS	AD_02.png	AD_02.puml	UC-15
activity-diagrams/	Book Service	AD_03.png	AD_03.puml	UC-04
activity-diagrams/	Cancel Booking	AD_04.png	AD_04.puml	UC-05
activity-diagrams/	Admin Garage Approval	AD_05.png	AD_05.puml	UC-21
context-diagram/	System Context Diagram	Context Diagram.drawio.png	context-diagram.puml	Section 2.1
sequence-diagram/	Emergency SOS Full Flow	Emergency SOS Sequence diagram.drawio.png	Emergency SOS.puml	UC-06, UC-07
sequence-diagram/	Booking Flow	Booking Flow.drawio.png	Booking Flow.puml	UC-04, UC-05
sequence-diagram/	Mechanic GPS Tracking	Mechanic Tracking GPS.drawio.png	Mechanic Tracking.puml	UC-07

4. Functional Requirements

4.1 Overview

This section defines the functional requirements for the GarageGo platform. Each requirement follows the §6.2.2 format with ID, Title, Description, Priority, Source, and Verification method. Priority levels are defined as:

- **P1** — Must have (core functionality, system cannot operate without it)
- **P2** — Should have (important but not critical for launch)
- **P3** — Nice to have (future enhancement)

4.2 Functional Requirements

FR-01: Customer Registration

Field	Detail
ID	FR-01
Title	Customer Registration
Description	The system shall allow a new customer to register by providing full name, phone number, email address, NIC number, and password. The system shall verify the phone number via OTP (Twilio) and verify the email address before activating the account.
Priority	P1
Source	Interview #9 Finding 1; UC-Registration
Verification	Integration test: register a new user with valid credentials, verify OTP is sent via Twilio, confirm email verification link is sent, confirm account is activated after both verifications.

FR-02: Vehicle Registration

Field	Detail
ID	FR-02
Title	Vehicle Registration
Description	The system shall allow a registered customer to add one or more vehicles to their account. Each vehicle record shall include vehicle type (two-wheel, four-wheel, or heavy), brand, model, year, fuel type, transmission type, registration number, and optional notes.
Priority	P1
Source	Interview #9 Finding 1; UC-02
Verification	Functional test: log in as customer, add a vehicle with all required fields, verify vehicle appears in the customer's vehicle list with correct details.

FR-03: Nearby Garage Search

Field	Detail
ID	FR-03
Title	Nearby Garage Search
Description	The system shall display nearby approved garages on a map and in a list based on the customer's current GPS location. The customer shall be able to filter results by vehicle type, services offered, ratings, operating hours, and current availability.
Priority	P1
Source	Survey #13 Q3; UC-02
Verification	Functional test: enable location, open garage search, verify garages within the defined radius are displayed on map and list. Apply each filter and verify results update correctly.

FR-04: Booking Creation

Field	Detail
ID	FR-04
Title	Booking Creation
Description	The system shall allow a customer to create a booking by selecting a vehicle, choosing a garage, selecting a service type, choosing a date and time, and adding optional notes. The system shall check garage capacity before confirming the booking slot.
Priority	P1
Source	UC-02; Interview #9 Finding 2
Verification	Functional test: create a booking with valid inputs, verify booking is stored with status "pending", verify garage receives a push notification. Attempt booking when no slots are available and verify HTTP 409 is returned.

FR-05: Emergency SOS Dispatch

Field	Detail
ID	FR-05
Title	Emergency SOS Dispatch
Description	The system shall allow a customer to trigger an emergency SOS request by selecting their affected vehicle, describing the issue, and sharing their GPS location. The system shall identify all eligible nearby garages filtered by zone and capacity and broadcast a live alert to all of them simultaneously via Socket.io.
Priority	P1
Source	Interview #9 Finding Q8; UC-01
Verification	Integration test: trigger SOS from customer app, verify emergency_request is inserted in DB with status "searching", verify all eligible garages receive the broadcast within 2 seconds.

FR-06: First-Accept-Wins Emergency Assignment

Field	Detail
ID	FR-06
Title	First-Accept-Wins Emergency Assignment
Description	The system shall assign an emergency request to the first garage that accepts it. The assignment shall use a SELECT FOR UPDATE database lock to prevent race conditions when multiple garages accept simultaneously. All other garages shall be notified that the emergency has already been assigned.
Priority	P1
Source	UC-01; UC-03
Verification	Concurrency test: simulate two garages accepting the same emergency simultaneously, verify only one is assigned and the DB lock prevents duplicate assignments.

FR-07: Mechanic GPS Tracking

Field	Detail
ID	FR-07
Title	Mechanic GPS Tracking
Description	The system shall allow the customer to track the assigned mechanic's real-time GPS location on a map during an active emergency. The mechanic app shall transmit location coordinates every 5 seconds via Socket.io. The system shall display the mechanic's estimated time of arrival and update it continuously.
Priority	P1
Source	UC-06; Interview #9 Finding 6
Verification	Integration test: start an emergency, have mechanic app emit location every 5 seconds, verify customer app receives location updates and ETA is displayed correctly. Simulate signal loss for 30 seconds and verify "Tracking paused" is shown.

FR-08: Garage Capacity Management

Field	Detail
ID	FR-08
Title	Garage Capacity Management
Description	The system shall allow a garage owner to set the maximum number of vehicles they can service simultaneously for each vehicle category (two-wheel, four-wheel, heavy). The system shall enforce this limit when accepting bookings and emergency requests.
Priority	P1
Source	UC-02; UC-03
Verification	Functional test: set capacity to 1 for four-wheel vehicles, create a booking that fills the slot, attempt a second booking for the same time slot and verify it is rejected with HTTP 409.

FR-09: Booking State Machine

Field	Detail
ID	FR-09
Title	Booking State Machine
Description	The system shall manage booking status transitions in the following order: pending → accepted → in_service → completed → paid. The system shall also support the cancelled state reachable from pending or accepted. Each transition shall trigger appropriate notifications to the customer and garage.
Priority	P1
Source	UC-02; UC-05
Verification	State machine test: walk a booking through each valid state transition and verify the DB status updates correctly. Attempt invalid transitions and verify they are rejected.

FR-10: Garage Registration

Field	Detail
ID	FR-10
Title	Garage Registration
Description	The system shall allow a garage owner to register their garage by providing business name, address, registration number, phone number, working hours, accepted vehicle types, services offered, location coordinates, emergency service zone, service capacity, and photos. The registration shall be submitted to an administrator for approval before the garage becomes visible to customers.
Priority	P1
Source	Interview #9 Finding 1; UC-04
Verification	Functional test: submit garage registration, verify status is "pending" and not visible to customers. Admin approves the garage, verify status changes to "approved" and garage appears in customer search results.

FR-11: Admin Garage Approval and Suspension

Field	Detail
ID	FR-11
Title	Admin Garage Approval and Suspension
Description	The system shall provide the administrator with the ability to review pending garage registrations and approve or reject them. The administrator shall also be able to suspend an approved garage if fraudulent activity or policy violations are detected. Suspended garages shall not appear in search results or receive new bookings.
Priority	P1
Source	UC-07
Verification	Functional test: admin approves a pending garage and verifies it becomes visible. Admin suspends an approved garage and verifies it disappears from search and cannot accept new bookings.

FR-12: Service History

Field	Detail
ID	FR-12
Title	Service History
Description	The system shall maintain a complete service history for each registered vehicle. Each service record shall include the garage name, date of service, services performed, invoice details, and payment status. The customer shall be able to view the full history per vehicle from within the app.
Priority	P1
Source	Interview #9 Finding Q5; UC-02
Verification	Functional test: complete a service booking, verify the service record is stored against the correct vehicle, open vehicle service history and confirm all fields are displayed correctly.

FR-13: Rating and Review

Field	Detail
ID	FR-13
Title	Rating and Review
Description	The system shall prompt the customer to submit a rating (1–5 stars) and an optional written review after each completed service. Reviews shall be displayed on the garage profile. A garage's average rating shall be calculated from all submitted reviews and updated in real time.
Priority	P2
Source	Survey #13 Q10; UC-02
Verification	Functional test: complete a booking, submit a rating and review, verify the review appears on the garage profile and the average rating is recalculated correctly.

FR-14: Push Notifications

Field	Detail
ID	FR-14
Title	Push Notifications
Description	The system shall send push notifications to the relevant app via the External Messaging System for the following events: new booking request (garage), booking accepted or rejected (customer), emergency broadcast (garage), emergency assigned (customer), mechanic arrived (customer), and booking status updates.
Priority	P1
Source	Interview #9 Finding 1; Survey #13 Q9
Verification	Integration test: trigger each notification event and verify the correct push notification is received by the correct recipient within 5 seconds.

FR-15: Mechanic Assignment and Dispatch

Field	Detail
ID	FR-15
Title	Mechanic Assignment and Dispatch
Description	The system shall allow the garage owner to assign a mechanic to an accepted emergency request. The assigned mechanic shall receive the job details in the mechanic app including the customer's GPS location and vehicle information. The mechanic shall be able to open navigation in Google Maps or Waze directly from the app.
Priority	P1
Source	UC-03; Interview #9 Finding 6
Verification	Functional test: accept an emergency, assign a mechanic, verify the mechanic app displays the job and customer location, verify the navigation link opens correctly.

FR-16: Complaint Submission

Field	Detail
ID	FR-16
Title	Complaint Submission
Description	The system shall allow customers to submit complaints against a garage for issues including overcharging, poor workmanship, no-shows, or unprofessional behaviour. The complaint shall be routed to the administrator dashboard for review and resolution.
Priority	P2
Source	Survey #13; UC-07
Verification	Functional test: submit a complaint from the customer app, verify it appears in the admin dashboard with correct details and status "open".

5. Non-Functional Requirements

5.1 Overview

This section defines the non-functional requirements (NFRs) for the GarageGo platform across six categories: Performance, Security, Usability, Reliability, Scalability, and Maintainability. All NFRs are measurable with specific thresholds.

5.2 Performance

NFR-P1: API Response Time

Field	Detail
ID	NFR-P1
Title	API Response Time
Requirement	The system shall respond to 95% of REST API requests within 2 seconds under a load of 50 concurrent users.
Threshold	≤ 2 seconds for 95th percentile response time at 50 concurrent users
Verification	Load test using Apache JMeter with 50 virtual users, measure p95 response time across all major endpoints.

NFR-P2: Emergency Broadcast Latency

Field	Detail
ID	NFR-P2
Title	Emergency Broadcast Latency
Requirement	The system shall deliver an emergency SOS broadcast to all eligible garages within 2 seconds of the customer triggering the SOS.
Threshold	≤ 2 seconds from POST /emergency to Socket.io delivery at all eligible garage clients
Verification	Integration test: measure time from SOS trigger to Socket.io event receipt at garage app using timestamped logs.

NFR-P3: Real-Time Location Update Frequency

Field	Detail
ID	NFR-P3
Title	Real-Time Location Update Frequency
Requirement	The system shall relay mechanic GPS location updates to the customer within 1 second of receiving the update from the mechanic app, maintaining a maximum update interval of 6 seconds end-to-end.
Threshold	≤ 1 second relay delay; ≤ 6 seconds total end-to-end interval
Verification	Integration test: measure timestamp difference between mechanic emit and customer receive events during active tracking.

5.3 Security

NFR-S1: JWT Authentication

Field	Detail
ID	NFR-S1
Title	JWT Authentication
Requirement	All API endpoints except /auth/register and /auth/login shall require a valid JSON Web Token. Tokens shall expire after 15 minutes. Refresh tokens shall expire after 7 days.
Threshold	100% of protected endpoints return HTTP 401 for missing or expired tokens
Verification	Security test: call each protected endpoint without a token and with an expired token, verify HTTP 401 is returned in all cases.

NFR-S2: Password Hashing

Field	Detail
ID	NFR-S2
Title	Password Hashing
Requirement	All user passwords shall be hashed using bcrypt with a minimum cost factor of 12 rounds before storage. Plain-text passwords shall never be stored or logged.
Threshold	bcrypt cost factor ≥ 12; zero plain-text password occurrences in DB or logs
Verification	Code review: verify bcrypt configuration. DB audit: confirm all password values are bcrypt hashes. Log scan: verify no plain-text passwords appear in application logs.

NFR-S3: Data Encryption in Transit

Field	Detail
ID	NFR-S3
Title	Data Encryption in Transit
Requirement	All communication between client apps and the API server shall use HTTPS with TLS 1.2 or higher. WebSocket connections shall use WSS protocol. HTTP requests shall be automatically redirected to HTTPS.
Threshold	100% of traffic over TLS 1.2+; zero unencrypted connections accepted
Verification	Security scan using SSL Labs, verify TLS version and certificate validity. Attempt plain HTTP connection and verify redirect to HTTPS.

5.4 Usability

NFR-U1: SOS Button Accessibility

Field	Detail
ID	NFR-U1
Title	SOS Button Accessibility
Requirement	The Emergency SOS button shall be visible on the customer app home screen without any scrolling. The button shall have a minimum tap target size of 48x48 dp as per Material Design guidelines.
Threshold	SOS button visible without scroll on all supported screen sizes; tap target ≥ 48x48 dp
Verification	UI test: open the app on the smallest supported screen size and verify the SOS button is visible without scrolling. Measure tap target dimensions.

NFR-U2: Registration Completion Time

Field	Detail
ID	NFR-U2
Title	Registration Completion Time
Requirement	A new customer shall be able to complete the full registration process including entering details, verifying phone OTP, and verifying email within 3 minutes under normal network conditions.
Threshold	≤ 3 minutes from app open to account activated
Verification	Usability test: time 5 new users completing registration on a standard 4G connection, verify average time is under 3 minutes.

NFR-U3: Offline Error Handling

Field	Detail
ID	NFR-U3
Title	Offline Error Handling
Requirement	The system shall display a clear user-friendly error message within 3 seconds when the customer app loses internet connectivity. The message shall not display raw error codes or stack traces.
Threshold	Error message displayed ≤ 3 seconds after connectivity loss; zero technical error codes shown to user
Verification	Functional test: disable network on device mid-session, verify user-friendly message appears within 3 seconds.

5.5 Reliability

NFR-R1: System Uptime

Field	Detail
ID	NFR-R1
Title	System Uptime
Requirement	The GarageGo API server shall maintain a minimum uptime of 99.5% during peak hours (7:00 AM – 9:00 PM Sri Lanka Standard Time), measured monthly.
Threshold	≥ 99.5% uptime during peak hours per month
Verification	Monitor uptime using Prometheus and Grafana over a 30-day period, verify monthly uptime report meets threshold.

NFR-R2: Webhook Failure Recovery

Field	Detail
ID	NFR-R2
Title	Webhook Failure Recovery
Requirement	If a webhook from the External Messaging System fails to arrive, the system shall trigger a reconciliation polling job within 10 minutes to verify and update the relevant booking or notification status.
Threshold	Reconciliation job triggered ≤ 10 minutes after missed webhook
Verification	Integration test: simulate a failed webhook delivery, verify reconciliation job runs within 10 minutes and status is corrected.

NFR-R3: Data Backup

Field	Detail
ID	NFR-R3
Title	Data Backup
Requirement	The PostgreSQL database shall be backed up automatically every 24 hours. Backups shall be retained for a minimum of 30 days. The system shall be recoverable from a backup within 2 hours in the event of data loss.
Threshold	Daily backups; 30-day retention; recovery time ≤ 2 hours
Verification	Operations test: verify automated backup runs daily, perform a restore from backup and measure recovery time.

5.6 Scalability

NFR-SC1: Concurrent Socket.io Connections

Field	Detail
ID	NFR-SC1
Title	Concurrent Socket.io Connections
Requirement	The Socket.io server shall support a minimum of 500 concurrent connections without degradation in message delivery time exceeding 1 second.
Threshold	≥ 500 concurrent connections; message delivery ≤ 1 second at full load
Verification	Load test: simulate 500 concurrent Socket.io clients, measure message delivery latency and verify it stays under 1 second.

NFR-SC2: Database Partitioning

Field	Detail
ID	NFR-SC2
Title	Database Partitioning
Requirement	The PostgreSQL bookings table shall support monthly horizontal partitioning to manage data growth. Queries on the current month's data shall return results within 500 milliseconds when the table contains over 100,000 records.
Threshold	Monthly partition support; query response ≤ 500ms at 100,000+ records
Verification	Performance test: insert 100,000 booking records, run common queries on current month partition and measure response time.

NFR-SC3: Horizontal API Scaling

Field	Detail
ID	NFR-SC3
Title	Horizontal API Scaling
Requirement	The GarageGo API server shall be deployable as multiple instances behind a load balancer. Adding a second instance shall increase throughput by at least 80% compared to a single instance under equivalent load.
Threshold	≥ 80% throughput increase when scaling from 1 to 2 instances
Verification	Load test: measure throughput with 1 instance, add a second instance under same load and verify throughput increase meets threshold.

5.7 Maintainability

NFR-M1: Unit Test Coverage

Field	Detail
ID	NFR-M1
Title	Unit Test Coverage
Requirement	The GarageGo backend codebase shall maintain a minimum unit test coverage of 70%, measured using Jest. Coverage reports shall be generated automatically as part of the CI/CD pipeline on every pull request.
Threshold	≥ 70% line coverage via Jest; coverage report generated on every PR
Verification	Run Jest with coverage flag in CI pipeline, verify coverage report shows ≥ 70% and pipeline fails if threshold is not met.

NFR-M2: Container Redeployment Time

Field	Detail
ID	NFR-M2
Title	Container Redeployment Time
Requirement	Each GarageGo system component (API server, Socket.io server, admin dashboard) shall be independently deployable via Docker. Redeploying any single component shall complete within 5 minutes without affecting other running components.
Threshold	Redeployment time ≤ 5 minutes per component; zero downtime for other components
Verification	Operations test: redeploy each Docker container independently, measure time from deployment start to healthy state, verify other components remain operational throughout.

NFR-M3: Code Documentation

Field	Detail
ID	NFR-M3
Title	Code Documentation
Requirement	All public API endpoints shall be documented using OpenAPI 3.0 specification. The API documentation shall be auto-generated and accessible at /api/docs when the server is running in development mode.
Threshold	100% of public endpoints documented in OpenAPI 3.0; /api/docs accessible in development
Verification	Code review: verify OpenAPI spec covers all endpoints. Start server in dev mode and confirm /api/docs loads with correct endpoint definitions.

6. Alternative Solutions & Feasibility Study

6.1 Alternative Solutions

This section evaluates alternative approaches to solving the same problem addressed by GarageGo and explains their limitations.

6.1.1 Manual System (Phone / WhatsApp / Walk-ins)

Description:

Customers directly contact garages via phone calls, WhatsApp messages, or physically visit workshops to request services.

Limitations:

- No real-time availability of garages
- Heavy reliance on word-of-mouth and personal contacts
- No structured booking or scheduling system
- Emergency breakdown handling is slow and uncoordinated
- No digital service history or records
- High risk of miscommunication and missed bookings

Conclusion:

This method is inefficient, unscalable, and unsuitable for real-time emergency or booking-based services.

6.1.2 Uber / PickMe Style Ride-Hailing Model (Generic Adaptation)

Description:

A ride-hailing style system similar to Uber or PickMe where users request a service and nearby providers accept in real time using a mobile platform.

Limitations:

- Designed for passenger transport, not vehicle repair workflows
- Does not support garage operations such as service bays or repair tracking
- No structured booking slots or scheduled maintenance system
- Limited support for multi-stage repair processes (diagnosis → repair → completion)
- No service history tracking or invoice-based workflow
- Emergency breakdown handling is not optimized for technical vehicle issues

Conclusion:

While this model provides strong real-time dispatching, it is not suitable for garage-specific workflows. It lacks domain-specific features required for vehicle repair management.

6.1.3 GarageGo System (Proposed Solution)

Description:

A full-stack, real-time, multi-sided platform with mobile apps for customers, garage owners, mechanics, and an admin dashboard.

Key Strengths:

- Real-time emergency SOS dispatch system
- First-accept-wins mechanic/garage assignment
- Live GPS tracking of mechanics
- Structured booking and garage capacity management
- Digital service history and ratings system
- Scalable cloud-based architecture

Conclusion:

GarageGo is a domain-specific solution that extends ride-hailing concepts into vehicle repair and roadside assistance, providing a complete end-to-end ecosystem.

6.2 Feasibility Study

6.2.1 Technical Feasibility

GarageGo is technically feasible using modern and widely adopted technologies.

Technology Stack:

- Frontend: React Native (cross-platform mobile apps)
- Backend: Node.js + Express (REST APIs + Socket.io real-time communication)
- Database: PostgreSQL (relational data + partitioning support)
- Cache Layer: Redis (sessions, queues, real-time data)
- Maps: Google Maps API (location, routing, distance calculations)
- Notifications: Firebase Cloud Messaging + SMS OTP service

Strengths:

- Real-time communication achievable via Socket.io
- GPS tracking supported natively on mobile devices
- Stateless backend enables horizontal scaling
- Cloud deployment supported via Docker containers

Risks:

- GPS accuracy depends on device and network conditions
- Socket.io requires proper scaling strategy
- Dependency on third-party APIs (Maps, SMS, Notifications)

Conclusion:

Technically feasible with moderate implementation complexity and manageable risks.

6.2.2 Economic Feasibility

GarageGo is economically feasible for an academic MVP and scalable for future commercialization.

Cost Factors:

- Cloud hosting (API, database, storage)
- Google Maps API usage costs
- SMS OTP services (per message pricing)
- Push notification services (low-cost/free tier)
- Development effort (team-based academic project)

Cost Optimization:

- No payment gateway integration (avoids transaction fees and compliance costs)
- Use of open-source technologies (Node.js, PostgreSQL, Redis)
- Cross-platform development reduces development cost

Future Revenue Options:

- Commission per completed booking
- Featured garage listings
- Subscription-based analytics for garages

Conclusion:

Low-cost MVP with strong potential for future monetization.

6.2.3 Operational Feasibility

GarageGo is operationally feasible in the Sri Lankan context.

User Side:

- High smartphone usage among target users
- Familiarity with apps like Uber, PickMe, WhatsApp
- Strong demand for faster garage discovery and emergency support

Garage Side:

- Replaces manual logbooks and phone-based booking
- Improves customer reach and reduces idle capacity
- Requires minimal training for adoption

Challenges:

- Resistance from traditional garage owners
- Need for consistent availability updates
- Dependence on user discipline for accurate data

Conclusion:

Operationally feasible with moderate onboarding effort.

6.2.4 Schedule Feasibility

The project is feasible within the academic timeline.

Estimated Timeline:

- Requirements & SRS: Completed
- System Design & UML: Completed
- Backend Development: 2–3 weeks
- Mobile App Development: 3–4 weeks
- Integration & Testing: 1–2 weeks
- Final Documentation: 1 week

Risks:

- Real-time system debugging (Socket.io + GPS sync)
- Third-party API integration delays
- Multi-device testing complexity

Conclusion:
Achievable within semester timeframe with proper task distribution.

6.3 Recommendation

GarageGo is recommended as the final solution.

Justification:

- Only solution supporting real-time emergency SOS dispatch
- Domain-specific workflow for vehicle repair and roadside assistance
- Combines booking, emergency response, and service tracking
- Improves both customer experience and garage efficiency
- Scalable architecture suitable for future expansion

Why alternatives are rejected:

- Manual systems are slow, unstructured, and non-scalable
- Uber/PickMe-style systems are not designed for garage repair workflows

Final Decision:
GarageGo is the most complete, scalable, and practical solution for the problem domain.

6.4 Full SRS Review & Change Log

6.4.1 Review Summary

Sections 1–5 were reviewed to ensure:

- Consistent terminology across all sections
- Alignment between use cases and functional requirements
- Proper traceability between diagrams and requirements
- Completeness of non-functional requirements
- Removal of placeholders and incomplete content

6.4.2 Remaining Assumptions

- Third-party APIs (Google Maps, SMS) remain stable
- Users have reliable mobile internet access
- Garage owners actively maintain availability updates

6.5 Final Submission Status

SRS Version: v0.1 (Final)

Sections Completed: 1–6

Diagrams: Included and referenced

Requirements: Fully defined and traceable

Final Output File

GarageGo-SRS-Final

Definition of Done Checklist

- Alternative solutions analyzed (Manual vs Uber/PickMe vs GarageGo)
- Technical, Economic, Operational, Schedule feasibility completed
- Recommendation clearly justified
- Full SRS reviewed for consistency
- Change log included
- Final document ready for submission
